



Heaven's Light is Our Guide

Department of Computer Science & Engineering
Rajshahi University of Engineering & Technology, Bangladesh

Course Code: CSE 2103
Course Title: Digital Logic Design

Presented by,
Prof. Dr. Boshir Ahmed
Md. Zahirul Islam

Canonical and Standard Forms

- We need to consider formal techniques for the simplification of Boolean functions.
 - Identical functions will have exactly the same canonical form.
 - Minterms and Maxterms
 - Sum-of-Minterms and Product-of- Maxterms
 - Product and Sum terms
 - Sum-of-Products (SOP) and Product-of-Sums (POS)

Definitions

- *Literal*: A variable or its complement
- *Product term*: literals connected by $\cdot$
- *Sum term*: literals connected by $+$
- *Minterm*: a product term in which all the variables appear exactly once, either complemented or uncomplemented
- *Maxterm*: a sum term in which all the variables appear exactly once, either complemented or uncomplemented

Minterm

- Represents exactly one combination in the truth table.
- Denoted by m_j , where j is the decimal equivalent of the minterm's corresponding binary combination (b_j).
- A variable in m_j is complemented if its value in b_j is 0, otherwise is uncomplemented.
- Example: Assume 3 variables (A,B,C), and $j=3$. Then, $b_j = 011$ and its corresponding minterm is denoted by $m_j = A'BC$

Maxterm

- Represents exactly one combination in the truth table.
- Denoted by M_j , where j is the decimal equivalent of the maxterm's corresponding binary combination (b_j).
- A variable in M_j is complemented if its value in b_j is 1, otherwise is uncomplemented.
- Example: Assume 3 variables (A,B,C), and $j=3$. Then, $b_j = 011$ and its corresponding maxterm is denoted by $M_j = A+B'+C'$

Truth Table notation for Minterms and Maxterms

- Minterms and Maxterms are easy to denote using a truth table.
- Example:
Assume 3 variables x, y, z (order is fixed)

x	y	z		Minterm	Maxterm
0	0	0		$x'y'z' = m_0$	$x+y+z = M_0$
0	0	1		$x'y'z = m_1$	$x+y+z' = M_1$
0	1	0		$x'yz' = m_2$	$x+y'+z = M_2$
0	1	1		$x'yz = m_3$	$x+y'+z' = M_3$
1	0	0		$xy'z' = m_4$	$x'+y+z = M_4$
1	0	1		$xy'z = m_5$	$x'+y+z' = M_5$
1	1	0		$xyz' = m_6$	$x'+y'+z = M_6$
1	1	1		$xyz = m_7$	$x'+y'+z' = M_7$

Canonical Forms (Unique)

- Any Boolean function $F()$ can be expressed as a *unique* **sum** of **minterms** and a unique **product** of **maxterms** (under a fixed variable ordering).
- In other words, every function $F()$ has two canonical forms:
 - Canonical Sum-Of-Products (sum of minterms)
 - Canonical Product-Of-Sums (product of maxterms)

Canonical Forms (cont.)

- Canonical Sum-Of-Products:
The minterms included are those m_j such that $F(\) = 1$ in row j of the truth table for $F(\)$.
- Canonical Product-Of-Sums:
The maxterms included are those M_j such that $F(\) = 0$ in row j of the truth table for $F(\)$.

Example

- Truth table for $f_1(a,b,c)$ at right
- The canonical sum-of-products form for f_1 is

$$f_1(a,b,c) = m_1 + m_2 + m_4 + m_6$$

$$= a'b'c + a'bc' + ab'c' + abc'$$
- The canonical product-of-sums form for f_1 is

$$f_1(a,b,c) = M_0 \cdot M_3 \cdot M_5 \cdot M_7$$

$$= (a+b+c) \cdot (a+b'+c') \cdot (a'+b+c') \cdot (a'+b'+c')$$
- Observe that: $m_j = M_j'$

a	b	c	f_1
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Shorthand: $\sum$ and $\prod$

- $f_1(a,b,c) = \sum m(1,2,4,6)$, where $\sum$ indicates that this is a sum-of-products form, and $m(1,2,4,6)$ indicates that the minterms to be included are m_1 , m_2 , m_4 , and m_6 .
- $f_1(a,b,c) = \prod M(0,3,5,7)$, where $\prod$ indicates that this is a product-of-sums form, and $M(0,3,5,7)$ indicates that the maxterms to be included are M_0 , M_3 , M_5 , and M_7 .
- Since $m_j = M_j'$ for any j ,
$$\sum m(1,2,4,6) = \prod M(0,3,5,7) = f_1(a,b,c)$$

Conversion Between Canonical Forms

- Replace $\sum$ with $\prod$ (or *vice versa*) and replace those j 's that appeared in the original form with those that do not.
- Example:

$$\begin{aligned}f_1(a,b,c) &= a'b'c + a'bc' + ab'c' + abc' \\&= m_1 + m_2 + m_4 + m_6 \\&= \sum(1,2,4,6) \\&= \prod(0,3,5,7) \\&= (a+b+c) \cdot (a+b'+c') \cdot (a'+b+c') \cdot (a'+b'+c')\end{aligned}$$

Standard Forms (NOT Unique)

- Standard forms are “*like*” canonical forms, except that not all variables need appear in the individual product (SOP) or sum (POS) terms.
- Example:
$$f_1(a,b,c) = a'b'c + bc' + ac'$$

is a *standard* sum-of-products form
- $$f_1(a,b,c) = (a+b+c) \cdot (b'+c') \cdot (a'+c')$$

is a *standard* product-of-sums form.

Conversion of SOP from standard to canonical form

- Expand *non-canonical* terms by inserting equivalent of 1 in each missing variable x:

$$(x + x') = 1$$

- Remove duplicate minterms

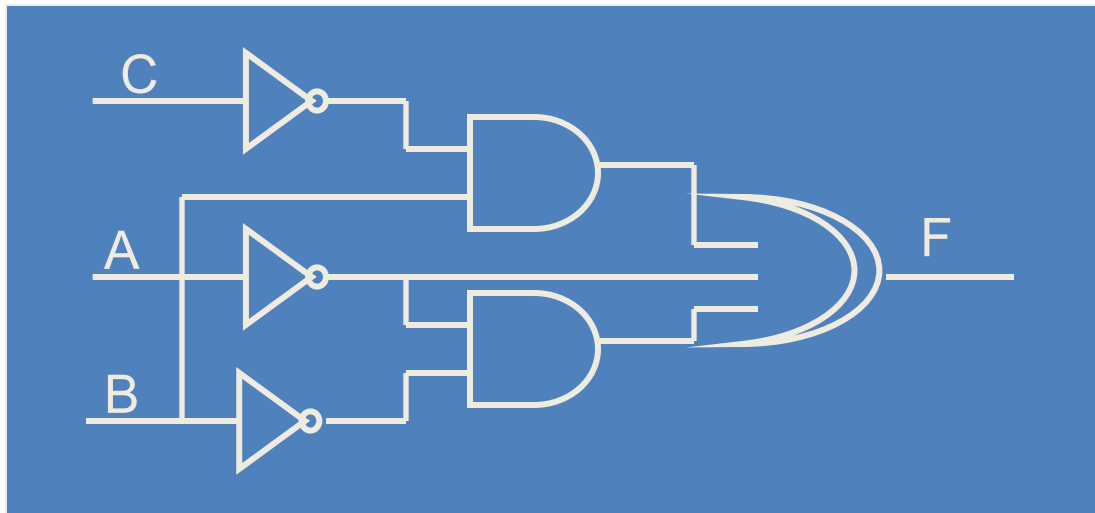
- $$\begin{aligned} f_1(a,b,c) &= a'b'c + bc' + ac' \\ &= a'b'c + (a+a')bc' + a(b+b')c' \\ &= a'b'c + abc' + a'bc' + abc' + ab'c' \\ &= a'b'c + abc' + a'bc + ab'c' \end{aligned}$$

Conversion of POS from standard to canonical form

- Expand noncanonical terms by adding 0 in terms of missing variables (e.g., $xx' = 0$) and using the distributive law
- Remove duplicate maxterms
- $f_1(a,b,c) = (a+b+c) \cdot (b'+c') \cdot (a'+c')$
 $= (a+b+c) \cdot (\textcolor{red}{a}a' + b' + c') \cdot (a' + \textcolor{red}{b}b' + c')$
 $= (a+b+c) \cdot (a+b'+c') \cdot (\textcolor{blue}{a'} + \textcolor{blue}{b'} + \textcolor{blue}{c'}) \cdot$
 $(a'+b+c') \cdot (\textcolor{blue}{a'} + \textcolor{blue}{b'} + \textcolor{blue}{c'})$
 $= (a+b+c) \cdot (a+b'+c') \cdot (a'+b'+c') \cdot (a'+b+c')$

Combinational Logic Circuit from Logic Function

- Consider function $F = A' + B \cdot C' + A' \cdot B'$
- A combinational logic circuit can be constructed to implement F , by appropriately connecting input signals and logic gates:
 - Circuit input signals $\square$ from function variables (A, B, C)
 - Circuit output signal $\square$ function output (F)
 - Logic gates $\square$ from logic operations

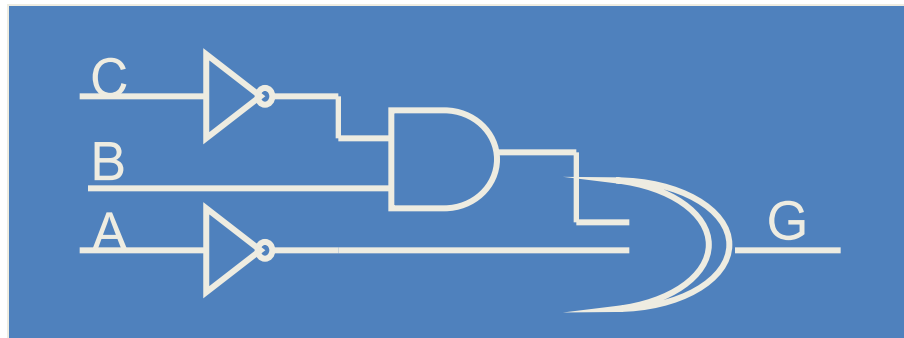
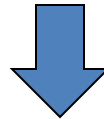
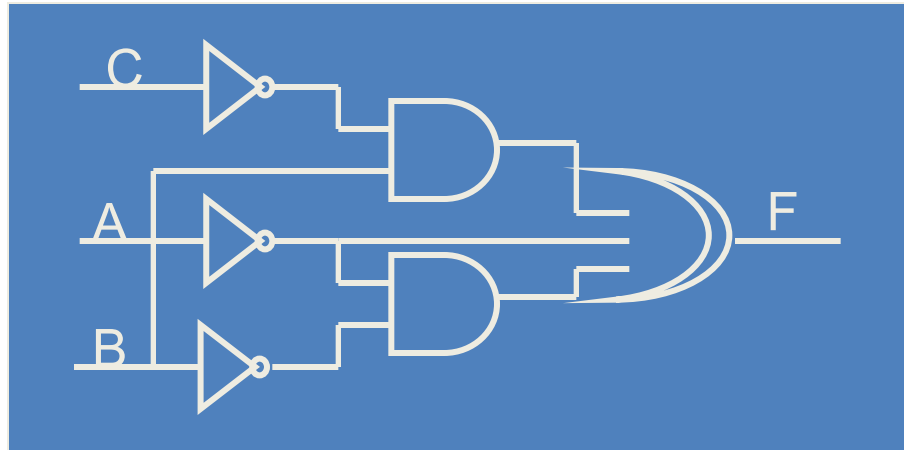


Combinational Logic Circuit from Logic Function (cont.)

- In order to design a cost-effective and efficient circuit, we must minimize the circuit's size (area) and propagation delay (time required for an input signal change to be observed at the output line)
- Observe the truth table of $F=A' + B \cdot C'$ + $A' \cdot B'$ and $G=A' + B \cdot C'$
- Truth tables for F and G are identical
□ same function
- Use G to implement the logic circuit (less components)

A	B	C	F	G
0	0	0	1	1
0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	1
1	1	1	0	0

Combinational Logic Circuit from Logic Function (cont.)



Circuit Simplification Methods

- Boolean algebra: greatly depends on inspiration and experience.
- Karnaugh map: systematic, step-by-step approach.

Algebraic Simplification

- Use the Boolean algebra theorems introduced in Lecture 4 to help simplify the expression for a logic circuit.
- Based on experience, often becomes a trial-and-error process.
- No easy way to tell whether a simplified expression is in its simplest form.

Two Essential Steps

- The original expression is put into the sum-of-products form by repeated application of DeMorgan's theorem and multiplication of terms.
- The product terms are checked for common factors, and factoring is performed whenever possible.

Examples 1- 4

Original	Simplified
$ABC + AB'(A'C)'$	$A(B' + C)$
$ABC + ABC' + AB'C$	$A(B + C)$
$A'C(A'BD)' + A'BC'D' + AB'C$	$B'C + A'D'(B + C)$
$(A' + B)(A + B + D)D'$	BD'

Examples 5 - 6

- $(A' + B)(A + B')$: equivalent form $A'B' + AB$
- $AB'C + A'BD + C'D'$: cannot be simplified further.

Boolean expressions-NOT unique

- Unlike truth tables, expressions representing a Boolean function are NOT unique.
- Example:
 - $F(x,y,z) = x' \cdot y' \cdot z' + x' \cdot y \cdot z' + x \cdot y \cdot z'$
 - $G(x,y,z) = x' \cdot y' \cdot z' + y \cdot z'$
- The corresponding truth tables for $F()$ and $G()$ are to the right. They are identical.
- Thus, $F() = G()$

x	y	z		F	G
0	0	0		1	1
0	0	1		0	0
0	1	0		1	1
0	1	1		0	0
1	0	0		0	0
1	0	1		0	0
1	1	0		1	1
1	1	1		0	0

Algebraic Manipulation

Example 7

- Boolean algebra is a useful tool for simplifying digital circuits.
- Why do it? Simpler can mean cheaper, smaller, faster.
- Example: Simplify $F = x'yz + x'yz' + xz$.

$$\begin{aligned} F &= x'yz + x'yz' + xz \\ &= x'y(z+z') + xz \\ &= x'y \cdot 1 + xz \\ &= x'y + xz \end{aligned}$$

Algebraic Manipulation (cont.)

Example 8

- Example: Prove

$$x'y'z' + x'yz' + xyz' = x'z' + yz'$$

- **Proof:**

$$\begin{aligned} & x'y'z' + x'yz' + xyz' \\ &= x'y'z' + x'yz' + x'yz' + xyz' \\ &= x'z'(y' + y) + yz'(x' + x) \\ &= x'z' \cdot 1 + yz' \cdot 1 \\ &= x'z' + yz' \end{aligned}$$

Complement of a Function

- The complement of a function is derived by interchanging ($\cdot$ and $+$), and (1 and 0), and complementing each variable.
- Otherwise, interchange 1s to 0s in the truth table column showing F.
- The *complement* of a function IS NOT THE SAME as the *dual* of a function.

Complementation

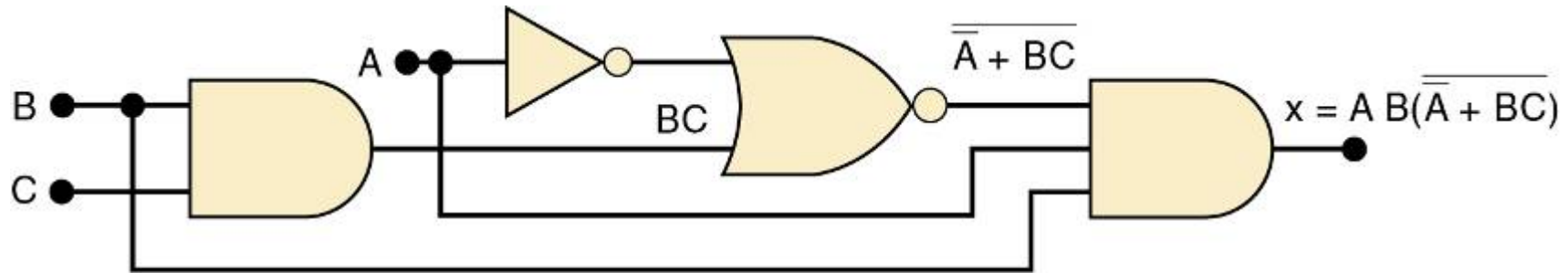
Example 9

- Find $G(x,y,z)$, the complement of $F(x,y,z) = xy'z' + x'yz$
- $G = F' = (xy'z' + x'yz)'$
 $= (xy'z')' \cdot (x'yz)'$ *DeMorgan*
 $= (x'+y+z) \cdot (x+y'+z')$ *DeMorgan* again
- Note: The complement of a function can also be derived by finding the function's *dual*, and then complementing all of the literals

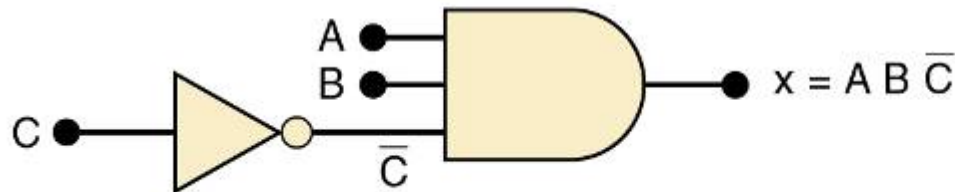
Simplifying Logic Circuits (Practice)

Example 10

- The circuits shown provide the same output
 - Circuit (b) is clearly less complex.



(a)



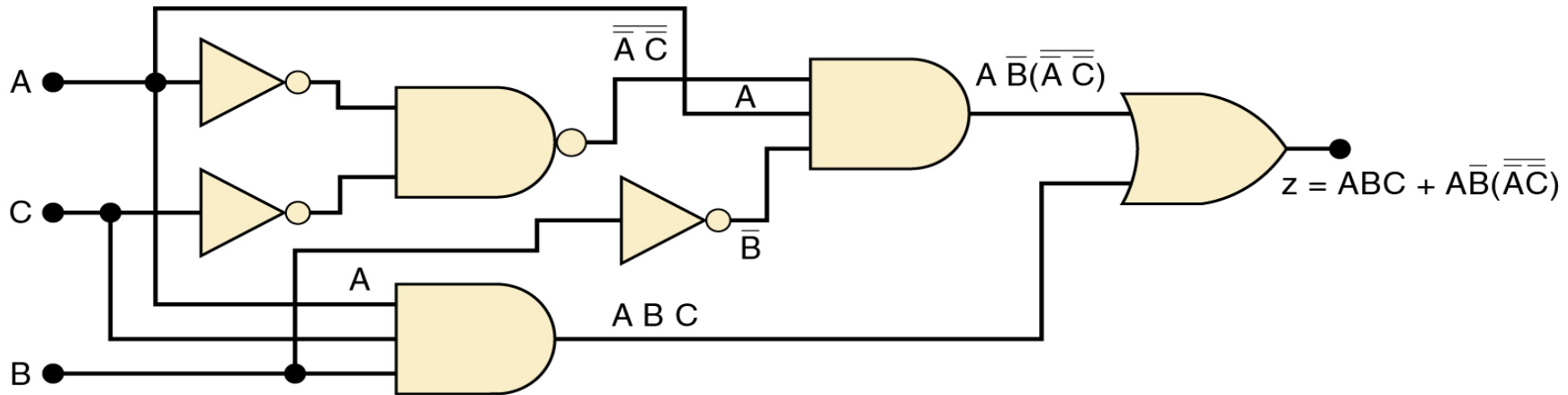
(b)

Logic circuits can be simplified using Boolean algebra and Karnaugh mapping.

Algebraic Simplification

Example 11

Simplify the logic circuit shown.



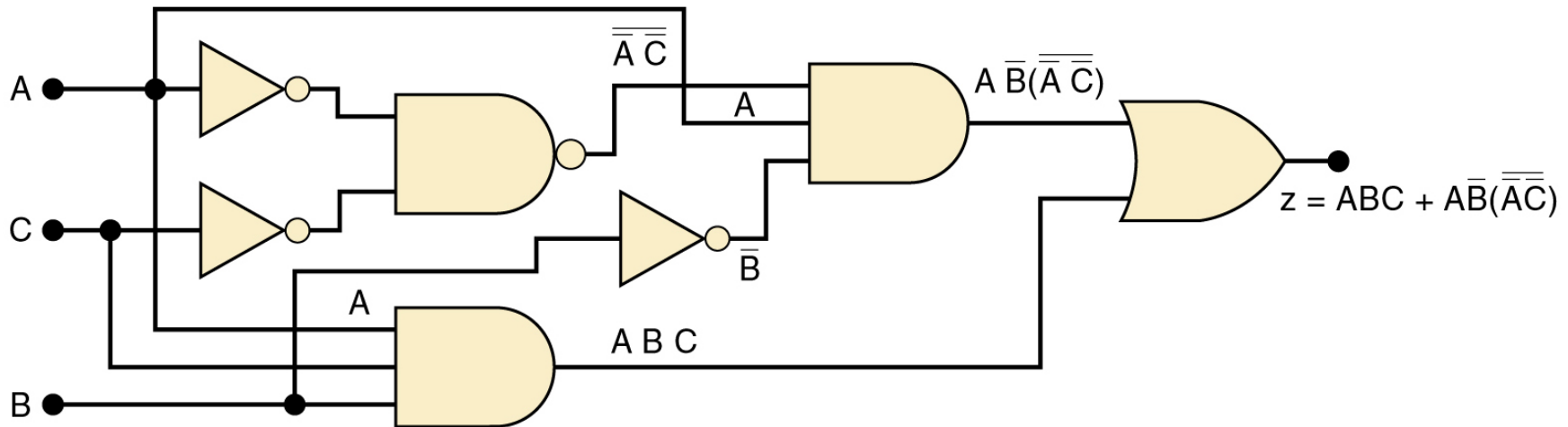
The first step is to determine the expression for the output: $z = ABC + AB\bar{A}\bar{C}$

Once the expression is determined, break down large inverter signs by DeMorgan's theorems & multiply out all terms.

$$\begin{aligned} z &= ABC + AB\bar{A}\bar{C} && [\text{theorem (17)}] \\ &= ABC + AB\bar{A}\bar{C} && [\text{cancel double inversions}] \\ &= ABC + AB\bar{A}\bar{C} && [\text{multiply out}] \\ &= ABC + AB + AB\bar{C} && [A \cdot A = A] \end{aligned}$$

Algebraic Simplification

Simplify the logic circuit shown.



Factoring—the first & third terms above have **AC** in common, which can be factored out:

$$z = AC(B + \overline{B}) + \overline{A} \overline{B}$$

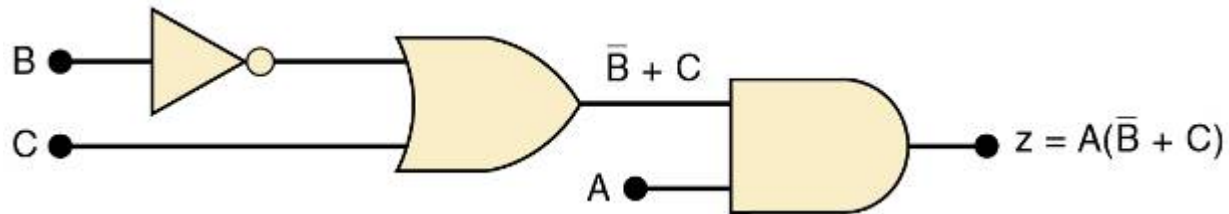
Since $B + B' = 1$, then...

$$\begin{aligned} z &= AC(1) + \overline{A} \overline{B} \\ &= AC + \overline{A} \overline{B} \end{aligned}$$

Factor out **A**, which results in...

Algebraic Simplification

Simplified logic circuit.



$$z = A(C + \bar{B})$$

Karnaugh Maps

- Karnaugh maps (K-maps) are *graphical* representations of Boolean functions.
- One ***map cell*** corresponds to a row in the truth table.
- Also, one map cell corresponds to a minterm or a maxterm in the Boolean expression
- Multiple-cell areas of the map correspond to standard terms.

Karnaugh Map Method

- A graphical device to simplify a logic expression.
- Generally work on examples with up to 4 input variables.
- From truth table to logic expression to K map.

Complete Simplification Process

- Step 1: Construct the K map and places 1s in those squares corresponding to the 1s in the truth table. Places 0s in the other squares.
- Step 2: Examine the map for adjacent 1s and loop those 1s which are not adjacent to any other 1s. (isolated 1s)
- Step 3: Look for those 1s which are adjacent to only one other 1. Loop any pair containing such a 1.
- Step 4: Loop any octet even when it contains some 1s that have already been looped.

Complete Simplification Process

- Step 5: Loop any quad that contains one or more 1s that have not already been looped, making sure to use the minimum number of loops.
- Step 6: Loop any pairs necessary to include any 1s have not already been looped, making sure to use the minimum number of loops.
- Step 7: Form the ORed sum of all the terms generated by each loop.

Filling K Map from Output Expression

- What to do when the desired output is presented as a Boolean expression instead of a truth table?
- Step 1: Convert the expression into SOP form.
- Step 2: For each product term in the SOP expression, place a 1 in each K-map square whose label contains the same combination of input values. Place a 0 in other squares.
- Example 9: $y = C'(A'B'D' + D) + AB'C + D'$

Two-Variable K-Map

Karnaugh Map (K-map)

- A visual representation of the function
- Same information as the truth table
- Easier to group minterms

x_1	x_2	f
0	0	0
0	1	1
1	0	0
1	1	1

(a) Truth table

		x_1	
		0	1
x_2	0	0	0
	1	1	1

(b) Karnaugh map

Karnaugh Map (K-map)

- A visual representation of the function
- Same information as the truth table
- Easier to group minterms

x_1	x_2	f
0	0	m_0
0	1	m_1
1	0	m_2
1	1	m_3

(a) Truth table

		x_1	
		0	1
x_2	0	m_0	m_2
	1	m_1	m_3

(b) Karnaugh map

Minterms

x_1	x_2	f
0	0	m_0
0	1	m_1
1	0	m_2
1	1	m_3

x_1	x_2	m_0	m_1	m_2	m_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Minterm Addition Example

x_1	x_2	f
0	0	0
0	1	1
1	0	0
1	1	1

x_1	x_2	m_0	m_1	m_2	m_3	$m_1 + m_3$
0	0	1	0	0	0	0
0	1	0	1	0	0	1
1	0	0	0	1	0	0
1	1	0	0	0	1	1

Minterm Addition Example

x_1	x_2	f
0	0	0
0	1	1
1	0	0
1	1	1

x_1	x_2	m_0	m_1	m_2	m_3	$m_1 + m_3$
0	0	1	0	0	0	0
0	1	0	1	0	0	1
1	0	0	0	1	0	0
1	1	0	0	0	1	1

$$\bar{x}_1 x_2 + x_1 x_2 = x_2$$

Minterm Addition Example

x_1	x_2	f
0	0	0
0	1	1
1	0	0
1	1	1

		x_1	
		0	1
x_2	0	0	0
	1	1	1

$$\bar{x}_1 x_2 + x_1 x_2 = x_2$$

Minterm Addition Example

x_1	x_2	f
0	0	0
0	1	1
1	0	0
1	1	1

		x_1	
		0	1
x_2	0	0	0
	1	1	1

x_2

$$\bar{x}_1 x_2 + x_1 x_2 = x_2$$

Another Grouping Example

		x_1	
		0	1
x_2	0	1	0
	1	0	0

m_0

		x_1	
		0	1
x_2	0	0	0
	1	1	0

m_1

Another Grouping Example

Diagram illustrating the addition of two 2x2 Karnaugh maps, m_0 and m_1 , to produce their sum $m_0 + m_1$.

The first map, m_0 , has the following values:

$x_2 \backslash x_1$	0	1
0	1	0
1	0	0

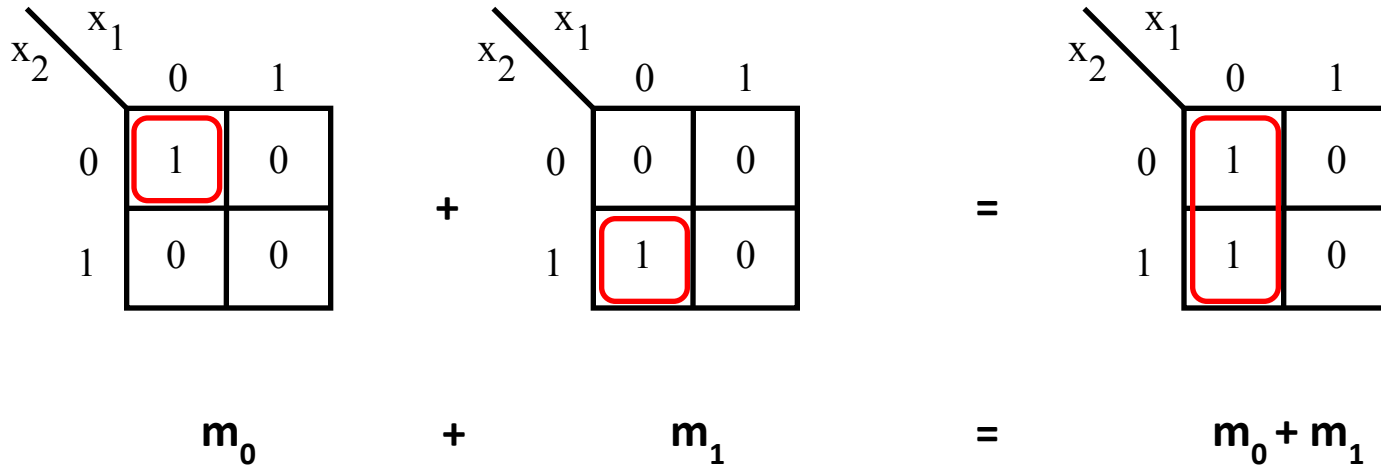
The second map, m_1 , has the following values:

$x_2 \backslash x_1$	0	1
0	0	0
1	1	0

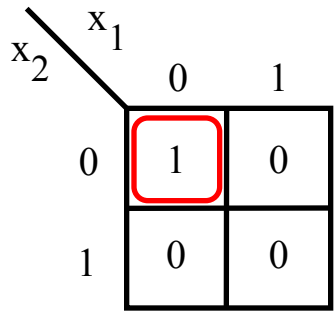
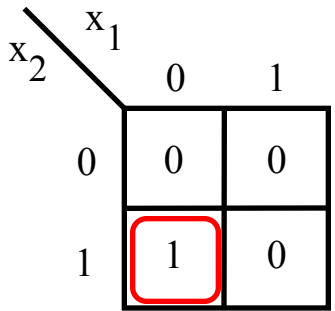
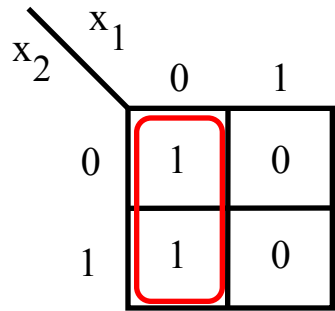
The sum map, $m_0 + m_1$, has the following values:

$x_2 \backslash x_1$	0	1
0	1	0
1	1	0

Another Grouping Example



Another Grouping Example

	+		=	
m_0	+	m_1	=	$m_0 + m_1$
$\overline{x_1} \overline{x_2}$	+	$\overline{x_1} x_2$	=	$\overline{x_1}$

Grouping Rules

- Group “1”s with rectangles
- Both sides must be a power of 2:
 - 1x1, 1x2, 2x1, 2x2, 1x4, 4x1, 2x4, 4x2, 4x4
- Can use/cover the same minterm more than once
- Can wrap around the edges of the map
- Some rules in selecting groups:
 - Try to use as few groups as possible to cover all “1”s.
 - For each group, try to make it as large as you can (i.e., if you can use a 2x2, don’t use a 2x1 even if that is enough).

Two-Variable K-map

x_1	x_2	f
0	0	m_0
0	1	m_1
1	0	m_2
1	1	m_3

(a) Truth table

		x_1	
		0	1
x_2	0	m_0	m_2
	1	m_1	m_3

(b) Karnaugh map

Step-By-Step Example

x_1	x_2	f
0	0	1
0	1	1
1	0	0
1	1	1

1. Draw The Map

x_1	x_2	f
0	0	1
0	1	1
1	0	0
1	1	1

		x_1	
		0	1
x_2	0		
	1		

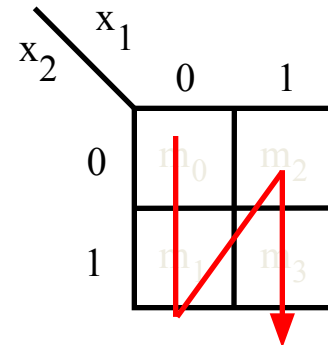
2. Fill The Map

	x_1	x_2	f
m_0	0	0	1
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1

x_1		
		x_2
0	1	
0	m_0	m_2
1	m_1	m_3

2. Fill The Map

	x_1	x_2	f
m_0	0	0	1
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1



2. Fill The Map

	x_1	x_2	f
m_0	0	0	1
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1

x_1		
		x_2
0	1	
0	1	0
1	1	1

3. Group

	x_1	x_2	f
m_0	0	0	1
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1

		x_1	
		0	1
x_2	0	1	0
	1	1	1

3. Group

	x_1	x_2	f
m_0	0	0	1
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1

$x_2 \backslash x_1$	0	1
0	1	0
1	1	1

3. Group

	x_1	x_2	f
m_0	0	0	1
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1

$x_2 \backslash x_1$	0	1
0	1	0
1	1	1

4. Write The Expression

x_1	x_2	f
0	0	1
0	1	1
1	0	0
1	1	1

		x_1	
x_2		0	1
0	1	1	0
1	1	1	1

4. Write The Expression

x_1	x_2	f
0	0	1
0	1	1
1	0	0
1	1	1

		x_1	
x_2		0	1
0	1	1	0
1	1	1	1

$$\bar{x}_1 + x_2$$

Writing The Expression

- Find which variable is constant

		x_1	
		0	1
x_2	0	1	0
	1	1	0

$\overline{x}_1$

Writing The Expression

- Find which variable is constant

		x_1	
		0	1
x_2	0	0	1
	1	0	1

x_1

Writing The Expression

- Find which variable is constant

		x_1	
		0	1
x_2	0	1	1
	1	0	0

$$\overline{x_2}$$

Writing The Expression

- Find which variable is constant

		x_1	
		0	1
x_2	0	0	0
	1	1	1

x_2

These are all valid groupings

		x_1	
		0	1
x_2	0	1	0
	1	0	0

$\bar{x}_1 \bar{x}_2$

		x_1	
		0	1
x_2	0	0	0
	1	1	0

$\bar{x}_1 x_2$

		x_1	
		0	1
x_2	0	0	1
	1	0	0

$x_1 \bar{x}_2$

		x_1	
		0	1
x_2	0	0	0
	1	0	1

$x_1 x_2$

These are all valid groupings

x_2	x_1	
	0	1
0	1	0
1	1	0

$\bar{x}_1$

x_2	x_1	
	0	1
0	0	1
1	0	1

x_1

x_2	x_1	
	0	1
0	1	1
1	0	0

$\bar{x}_2$

x_2	x_1	
	0	1
0	0	0
1	1	1

x_2

This one is valid too

		x_1	
		0	1
x_2	0	1	1
	1	1	1

In this case the result is the constant function 1.

Invalid Groupings

A Karnaugh map for variables x_1 and x_2 . The map is a 2x2 grid. The columns are labeled x_1 with values 0 and 1. The rows are labeled x_2 with values 0 and 1. The cells contain the following values:

$x_2 \backslash x_1$	0	1
0	1	0
1	0	1

A red line is drawn diagonally from the top-left cell (1) to the bottom-right cell (1), representing an invalid grouping because it does not follow the rules of Karnaugh map simplification (it is not a power of 2 in size and does not wrap around).

A Karnaugh map for variables x_1 and x_2 . The map is a 2x2 grid. The columns are labeled x_1 with values 0 and 1. The rows are labeled x_2 with values 0 and 1. The cells contain the following values:

$x_2 \backslash x_1$	0	1
0	0	1
1	1	0

A red line is drawn diagonally from the top-right cell (1) to the bottom-left cell (1), representing an invalid grouping because it does not follow the rules of Karnaugh map simplification (it is not a power of 2 in size and does not wrap around).

Can't group diagonally. Why?

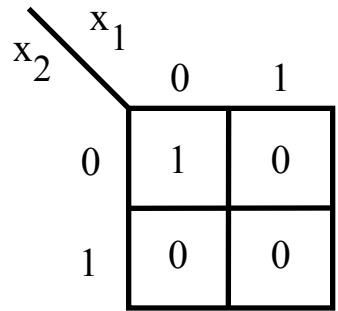
		x_1	
		0	1
x_2	0	1	0
	1	0	0

m_0

		x_1	
		0	1
x_2	0	0	0
	1	0	1

m_3

Can't group diagonally. Why?

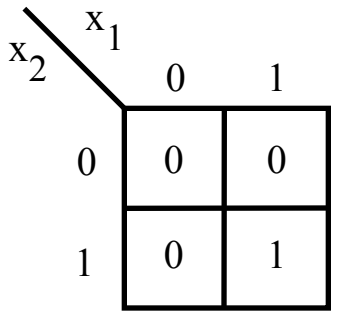


A 2x2 Karnaugh map for variables x_1 and x_2 . The columns are labeled 0 and 1 for x_1 , and the rows are labeled 0 and 1 for x_2 . The cells contain the values 1, 0, 0, and 0 respectively.

$x_2 \backslash x_1$	0	1
0	1	0
1	0	0

m_0

+

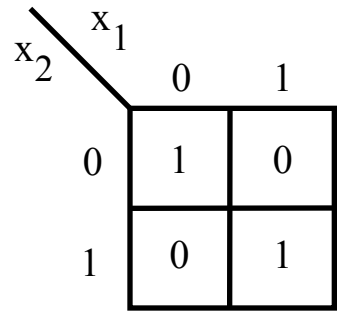


A 2x2 Karnaugh map for variables x_1 and x_2 . The columns are labeled 0 and 1 for x_1 , and the rows are labeled 0 and 1 for x_2 . The cells contain the values 0, 0, 0, and 1 respectively.

$x_2 \backslash x_1$	0	1
0	0	0
1	0	1

m_3

=

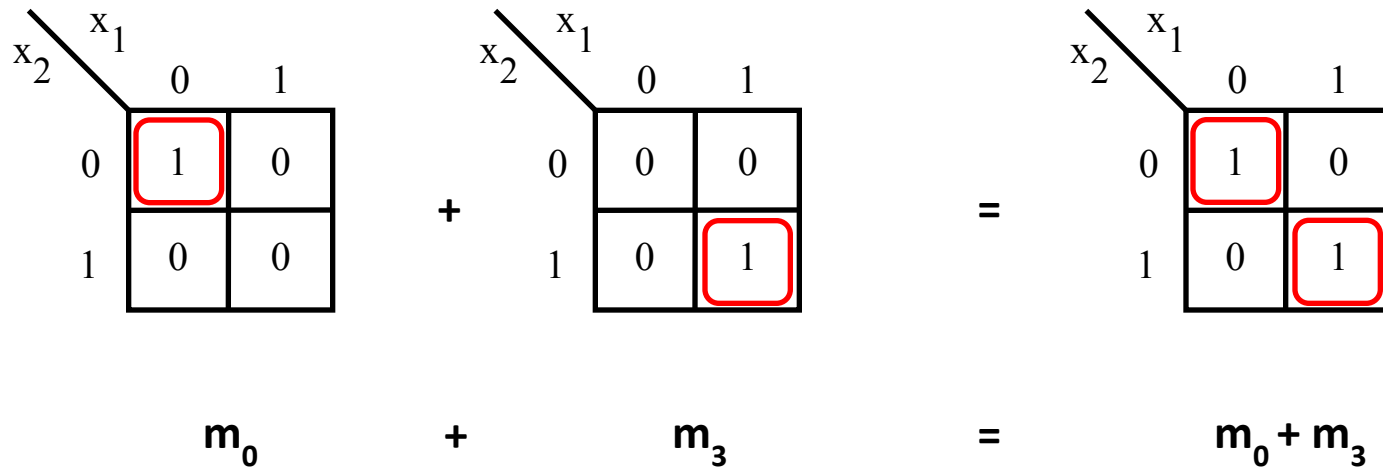


A 2x2 Karnaugh map for variables x_1 and x_2 . The columns are labeled 0 and 1 for x_1 , and the rows are labeled 0 and 1 for x_2 . The cells contain the values 1, 0, 0, and 1 respectively.

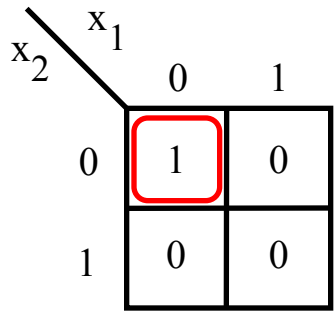
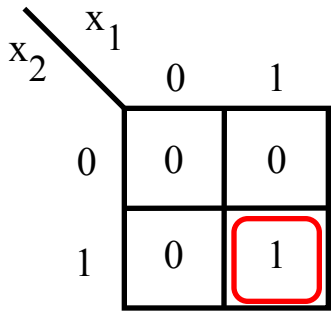
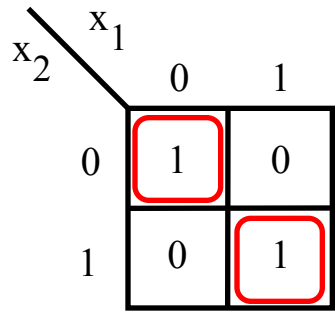
$x_2 \backslash x_1$	0	1
0	1	0
1	0	1

$m_0 + m_3$

Can't group diagonally. Why?



Can't group diagonally. Why?

	+		=	
m_0	+	m_3	=	$m_0 + m_3$
$\overline{x_1}\overline{x_2}$	+	x_1x_2	=	$\overline{x_1}\overline{x_2} + x_1x_2$

Three-Variable K-Map

Location of three-variable minterms

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

		$x_1 x_2$			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

(b) Karnaugh map

Location of three-variable minterms

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

		$x_1 x_2$			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

(b) Karnaugh map

Notice the placement of

- **Variables**
- **Binary pair values**
- **Minterms**

Location of three-variable minterms

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

		$x_1 x_2$			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

(b) Karnaugh map

Notice the placement of

- **Variables**
- **Binary pair values**
- **Minterms**

Location of three-variable minterms

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

		$x_1 x_2$			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

(b) Karnaugh map

Notice the placement of

- **Variables**
- **Binary pair values**
- **Minterms**

Gray Code

- Sequence of binary codes
- Two neighboring lines vary by only 1 bit

00

01

11

10

Gray Code

- Sequence of binary codes
- Two neighboring lines vary by only 1 bit

000
001
011
010
110
111
101
100

Gray Code

- Sequence of binary codes
- Two neighboring lines vary by only 1 bit

000
001
011
010
110
111
101
100

Gray Code

- Sequence of binary codes
- Two neighboring lines vary by only 1 bit

000
001
011
010
110
111
101
100

Gray Code

- Sequence of binary codes
- Two neighboring lines vary by only 1 bit

000
001
011
010
110
111
101
100

Gray Code

- Sequence of binary codes
- Two neighboring lines vary by only 1 bit

000
001
011
010
110
111
101
100

Gray Code

- Sequence of binary codes
- Two neighboring lines vary by only 1 bit

000
001
011
010
110
111
101
100

Gray Code

- Sequence of binary codes
- Two neighboring lines vary by only 1 bit

000
001
011
010
110
111
101
100

Gray Code

- Sequence of binary codes
- Two neighboring lines vary by only 1 bit

000
001
011
010
110
111
101
100

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

		$s \ x_1$			
		00	01	11	10
x_2	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

		$s \ x_1$			
		00	01	11	10
x_2	0	000	010	110	100
	1	001	011	111	101

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

		$s \ x_1$			
x_2		00	01	11	10
	0	000	010	110	100
	1	001	011	111	101

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

		$s \ x_1$			
		00	01	11	10
x_2	0	000	010	110	100
	1	001	011	111	101

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

		$s \ x_1$			
		00	01	11	10
x_2	0	000	010	110	100
	1	001	011	111	101

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

		$s \ x_1$			
		00	01	11	10
x_2	0	000	010	110	100
	1	001	011	111	101

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

		$s \ x_1$			
		00	01	11	10
x_2	0	000	010	110	100
	1	001	011	111	101

These two neighbors
differ only in the LAST bit

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

$s \ x_1$	00	01	11	10
x_2				
0	000	010	110	100
1	001	011	111	101

These two neighbors
differ only in the LAST bit

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

x_2	$s x_1$			
	00	01	11	10
0	000	010	110	100
1	001	011	111	101

These two neighbors
differ only in the FIRST bit

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

$s \ x_1$	00	01	11	10
x_2				
0	000	010	110	100
1	001	011	111	101

These four neighbors
differ in the FIRST and LAST bit

They are similar in their MIDDLE bit

Adjacency Rules

$x_3 \backslash x_1 x_2$					
		00	01	11	10
0	m_0	m_2	m_6	m_4	
1	m_1	m_3	m_7	m_5	



adjacent
columns

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

		$s \ x_1$			
		00	01	11	10
x_2	0	000	010	110	100
	1	001	011	111	101

These two neighbors
differ only in the FIRST bit

Gray Code & K-map

	s	x_1	x_2
m_0	0	0	0
m_1	0	0	1
m_2	0	1	0
m_3	0	1	1
m_4	1	0	0
m_5	1	0	1
m_6	1	1	0
m_7	1	1	1

$x_2 \backslash s x_1$	00	01	11	10
0	000	010	110	100
1	001	011	111	101

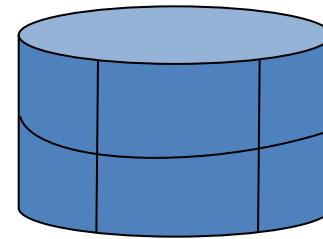
These four neighbors
differ in the FIRST and LAST bit

They are similar in their MIDDLE bit

Adjacency Rules

x_1x_2					
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

adjacent
columns

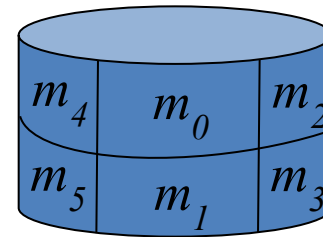


As if the K-map were
drawn on a cylinder

Adjacency Rules

x_1x_2					
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

adjacent
columns

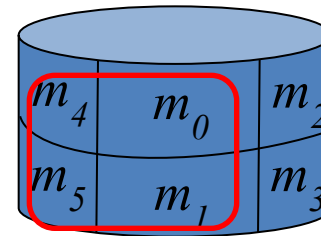


As if the K-map were
drawn on a cylinder

Adjacency Rules

x_1x_2		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

adjacent
columns

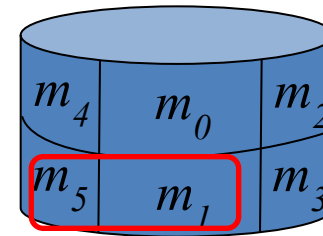


As if the K-map were
drawn on a cylinder

Adjacency Rules

x_1x_2		x_3			
		0	1		
	00	01	11	10	
0	m_0	m_2	m_6	m_4	
1	m_1	m_3	m_7	m_5	

adjacent columns



As if the K-map were drawn on a cylinder

Examples of Valid Groupings

Groupings and Expressions

		x_1x_2			
		00	01	11	10
x_3	0	1	0	0	0
	1	0	0	0	0

$\overline{x_1} \overline{x_2} \overline{x_3}$

		x_1x_2			
		00	01	11	10
x_3	0	0	1	0	0
	1	0	0	0	0

$\overline{x_1} x_2 \overline{x_3}$

		x_1x_2			
		00	01	11	10
x_3	0	0	0	1	0
	1	0	0	0	0

$x_1 x_2 \overline{x_3}$

		x_1x_2			
		00	01	11	10
x_3	0	0	0	0	1
	1	0	0	0	0

$\overline{x_1} \overline{x_2} x_3$

Groupings and Expressions

		x_1x_2			
		00	01	11	10
x_3	0	0	0	0	0
	1	1	0	0	0

$\overline{x_1} \overline{x_2} x_3$

		x_1x_2			
		00	01	11	10
x_3	0	0	0	0	0
	1	0	1	0	0

$\overline{x_1} x_2 \overline{x_3}$

		x_1x_2			
		00	01	11	10
x_3	0	0	0	0	0
	1	0	0	1	0

$x_1 x_2 x_3$

		x_1x_2			
		00	01	11	10
x_3	0	0	0	0	0
	1	0	0	0	1

$\overline{x_1} \overline{x_2} x_3$

Groupings and Expressions

		x_1x_2			
		00	01	11	10
x_3	0	1	0	0	0
	1	1	0	0	0

$\overline{x_1} \overline{x_2}$

		x_1x_2			
		00	01	11	10
x_3	0	0	1	0	0
	1	0	1	0	0

$\overline{x_1} x_2$

		x_1x_2			
		00	01	11	10
x_3	0	0	0	1	0
	1	0	0	1	0

$x_1 x_2$

		x_1x_2			
		00	01	11	10
x_3	0	0	0	0	1
	1	0	0	0	1

$x_1 \overline{x_2}$

Groupings and Expressions

		x_1x_2			
x_3		00	01	11	10
	0	1	1	0	0
	1	0	0	0	0

$\overline{x_1} \overline{x_3}$

		x_1x_2			
x_3		00	01	11	10
	0	0	1	1	0
	1	0	0	0	0

$x_2 \overline{x_3}$

		x_1x_2			
x_3		00	01	11	10
	0	0	0	1	1
	1	0	0	0	0

$x_1 \overline{x_3}$

		x_1x_2			
x_3		00	01	11	10
	0	1	0	0	1
	1	0	0	0	0

$\overline{x_2} \overline{x_3}$

Groupings and Expressions

x_1x_2					
x_3		00	01	11	10
0		0	0	0	0
1		1	1	0	0

$\overline{x_1} x_3$

x_1x_2					
x_3		00	01	11	10
0		0	0	0	0
1		0	1	1	0

$x_2 x_3$

x_1x_2					
x_3		00	01	11	10
0		0	0	0	0
1		0	0	1	1

$x_1 x_3$

x_1x_2					
x_3		00	01	11	10
0		0	0	0	0
1		1	0	0	1

$\overline{x_2} x_3$

Groupings and Expressions

x_1x_2		00	01	11	10
x_3					
0		1	1	0	0
1		1	1	0	0

$\overline{x_1}$

x_1x_2		00	01	11	10
x_3					
0		0	1	1	0
1		0	1	1	0

x_2

x_1x_2		00	01	11	10
x_3					
0		0	0	1	1
1		0	0	1	1

x_1

x_1x_2		00	01	11	10
x_3					
0		1	0	0	1
1		1	0	0	1

$\overline{x_2}$

Groupings and Expressions

x_1x_2					
x_3		00	01	11	10
0		1	1	1	1
1		0	0	0	0

$\overline{x_3}$

x_1x_2					
x_3		00	01	11	10
0		0	0	0	0
1		1	1	1	1

x_3

Groupings and Expressions

		x_1x_2			
		00	01	11	10
x_3	0	0	0	0	0
	1	0	0	0	0

0

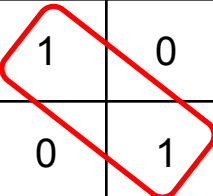
		x_1x_2			
		00	01	11	10
x_3	0	1	1	1	1
	1	1	1	1	1

1

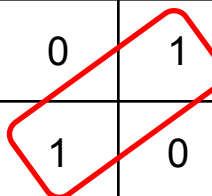
Examples of Invalid Groupings

Some Invalid Groupings

		x_1x_2			
		00	01	11	10
x_3	0	0	1	0	0
	1	0	0	1	0



		x_1x_2			
		00	01	11	10
x_3	0	0	0	1	0
	1	0	1	0	0



Can't group diagonally.

Some Invalid Groupings

		x_1x_2			
		00	01	11	10
x_3	0	1	1	1	0
	1	0	0	0	0

		x_1x_2			
		00	01	11	10
x_3	0	0	0	0	0
	1	0	1	1	1

Can't group three in a row.
Each side must be a power of 2.

Some Invalid Groupings

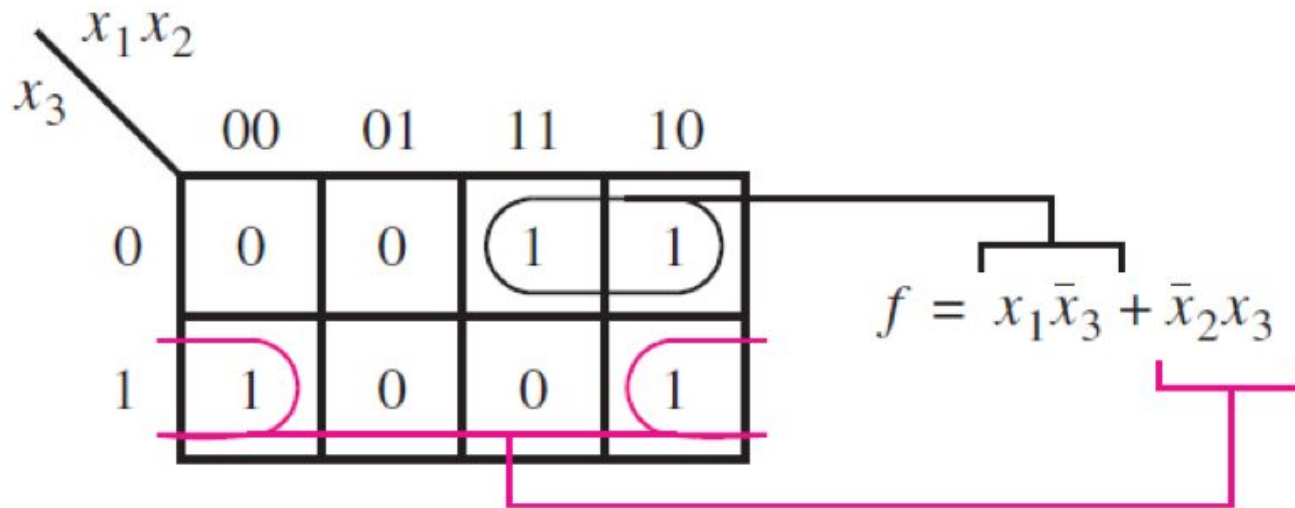
x_1x_2		00	01	11	10
x_3					
0		1	0	1	1
1		0	0	0	0

x_1x_2		00	01	11	10
x_3					
0		0	0	1	0
1		0	1	1	0

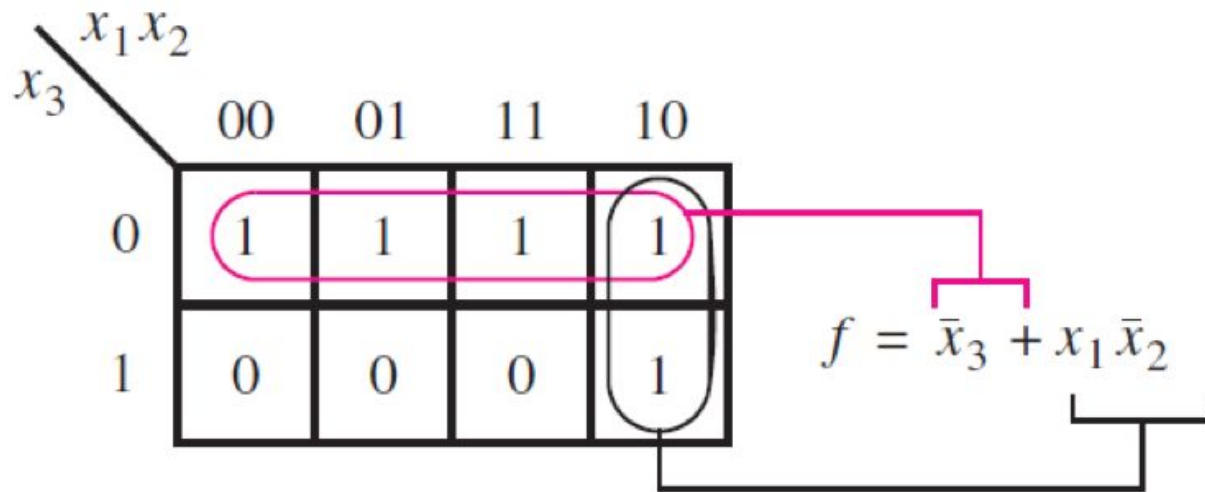
Can't group zeros and ones together.

Minimization Examples with 3-variable K-Maps

Examples of three-variable Karnaugh maps

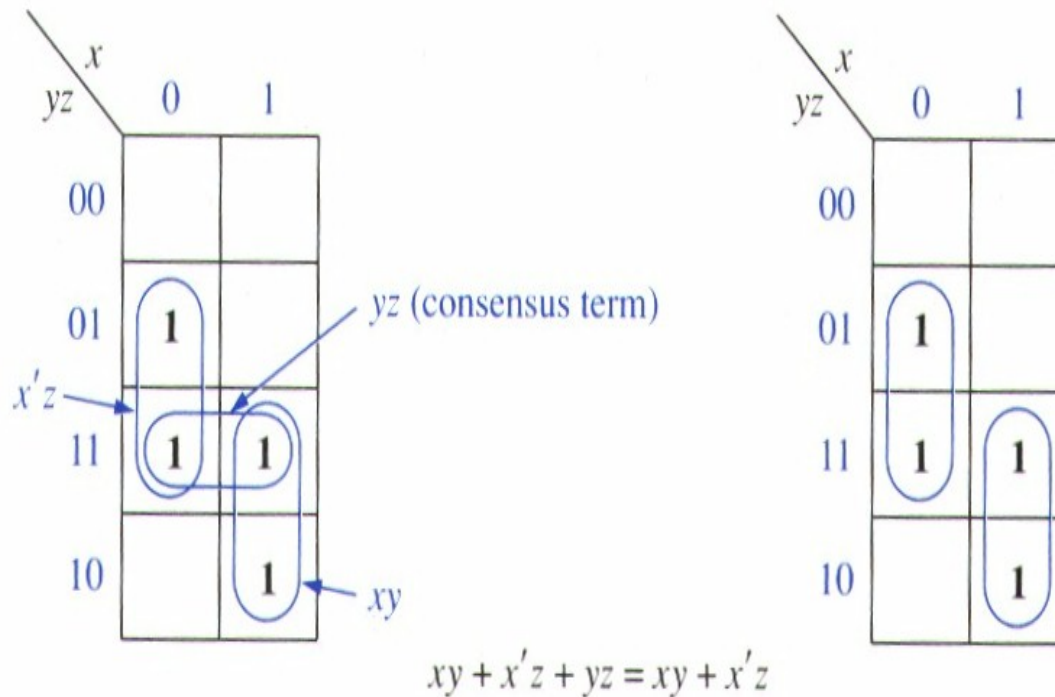


Examples of three-variable Karnaugh maps



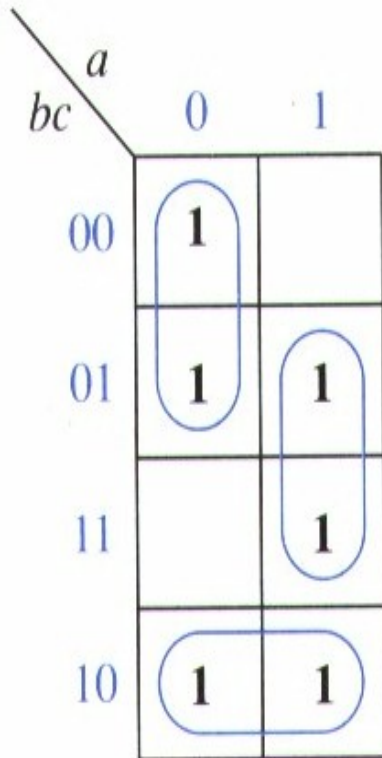
Redundant Terms

- If a term is covered by two other terms, that term is redundant. That is, it is a consensus term.
- Example: yz is the redundant.

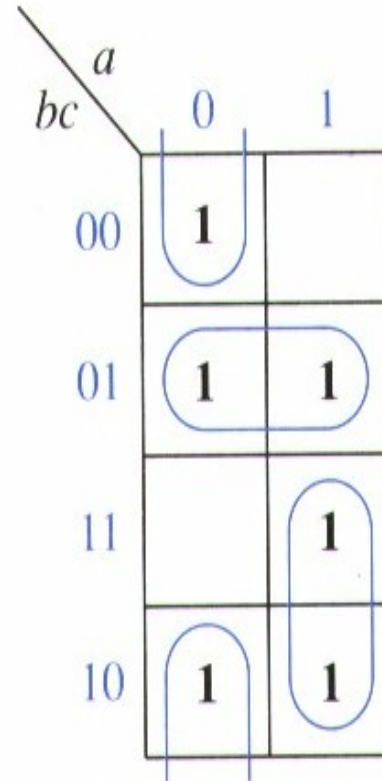


More Than Two Minimum Solutions

- $F = \sum m(0,1,2,5,6,7)$



$$F = a'b' + bc' + ac$$



$$F = a'c' + b'c + ab$$

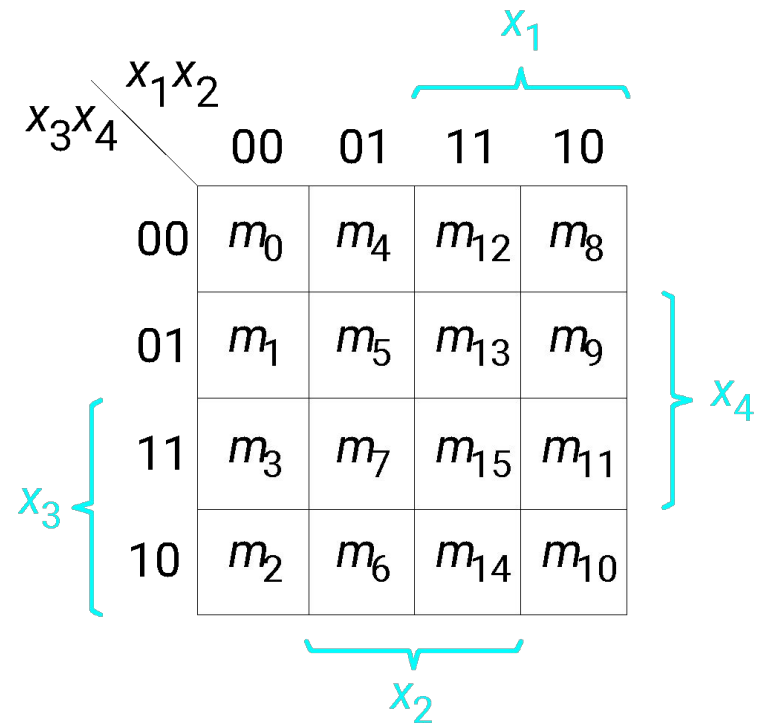
Four-Variable K-Map

A four-variable Karnaugh map

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
00	m_0	m_4	m_{12}	m_8	x_4
01	m_1	m_5	m_{13}	m_9	
11	m_3	m_7	m_{15}	m_{11}	
10	m_2	m_6	m_{14}	m_{10}	
x_3		x_2			

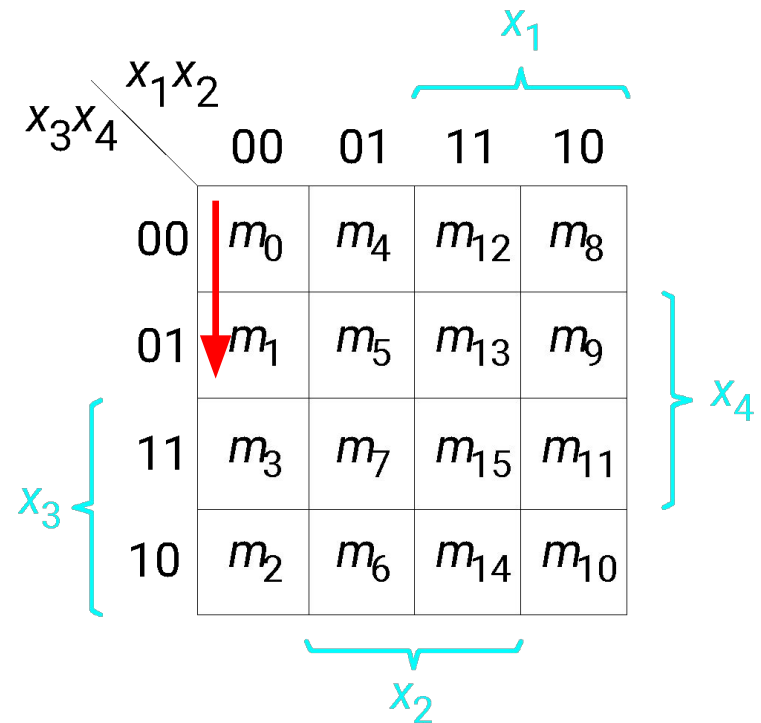
A four-variable Karnaugh map

x1	x2	x3	x4	
0	0	0	0	m0
0	0	0	1	m1
0	0	1	0	m2
0	0	1	1	m3
0	1	0	0	m4
0	1	0	1	m5
0	1	1	0	m6
0	1	1	1	m7
1	0	0	0	m8
1	0	0	1	m9
1	0	1	0	m10
1	0	1	1	m11
1	1	0	0	m12
1	1	0	1	m13
1	1	1	0	m14
1	1	1	1	m15



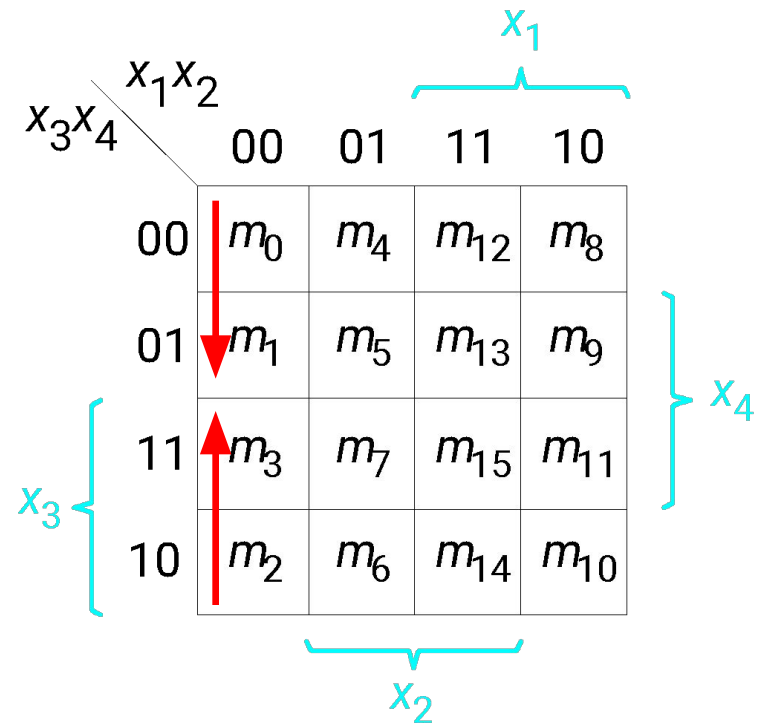
A four-variable Karnaugh map

x1	x2	x3	x4	
0	0	0	0	m0
0	0	0	1	m1
0	0	1	0	m2
0	0	1	1	m3
0	1	0	0	m4
0	1	0	1	m5
0	1	1	0	m6
0	1	1	1	m7
1	0	0	0	m8
1	0	0	1	m9
1	0	1	0	m10
1	0	1	1	m11
1	1	0	0	m12
1	1	0	1	m13
1	1	1	0	m14
1	1	1	1	m15



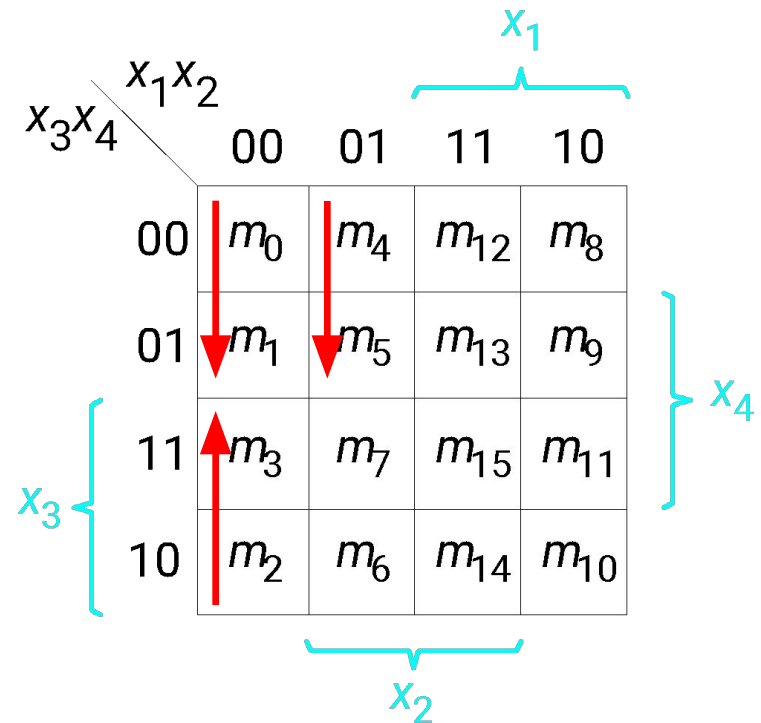
A four-variable Karnaugh map

x1	x2	x3	x4	
0	0	0	0	m0
0	0	0	1	m1
0	0	1	0	m2
0	0	1	1	m3
0	1	0	0	m4
0	1	0	1	m5
0	1	1	0	m6
0	1	1	1	m7
1	0	0	0	m8
1	0	0	1	m9
1	0	1	0	m10
1	0	1	1	m11
1	1	0	0	m12
1	1	0	1	m13
1	1	1	0	m14
1	1	1	1	m15



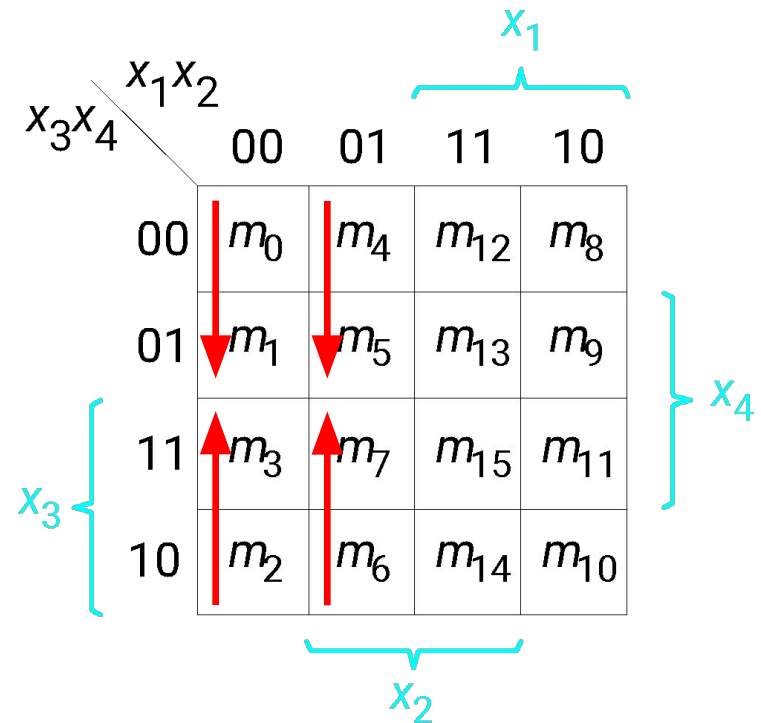
A four-variable Karnaugh map

x1	x2	x3	x4	
0	0	0	0	m0
0	0	0	1	m1
0	0	1	0	m2
0	0	1	1	m3
0	1	0	0	m4
0	1	0	1	m5
0	1	1	0	m6
0	1	1	1	m7
1	0	0	0	m8
1	0	0	1	m9
1	0	1	0	m10
1	0	1	1	m11
1	1	0	0	m12
1	1	0	1	m13
1	1	1	0	m14
1	1	1	1	m15



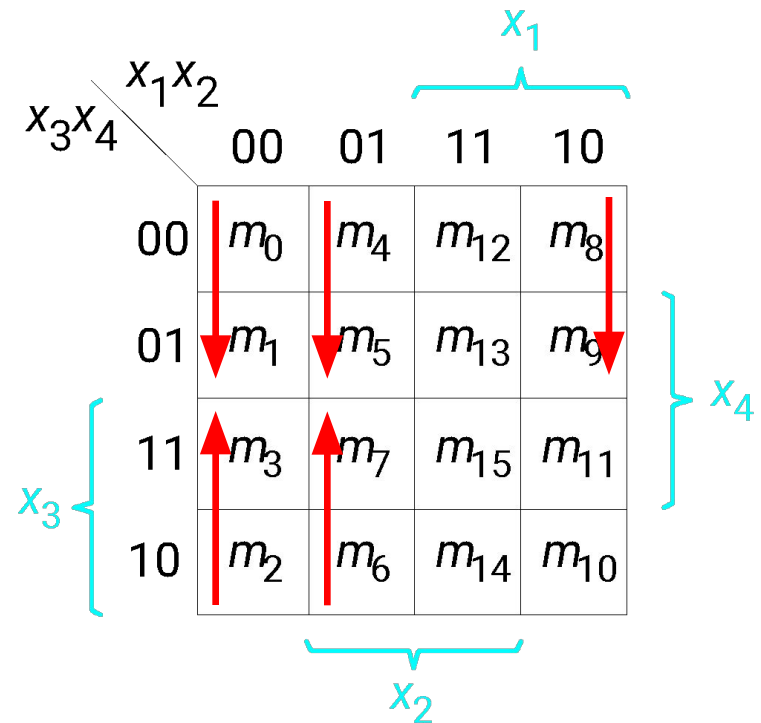
A four-variable Karnaugh map

x1	x2	x3	x4	
0	0	0	0	m0
0	0	0	1	m1
0	0	1	0	m2
0	0	1	1	m3
0	1	0	0	m4
0	1	0	1	m5
0	1	1	0	m6
0	1	1	1	m7
1	0	0	0	m8
1	0	0	1	m9
1	0	1	0	m10
1	0	1	1	m11
1	1	0	0	m12
1	1	0	1	m13
1	1	1	0	m14
1	1	1	1	m15



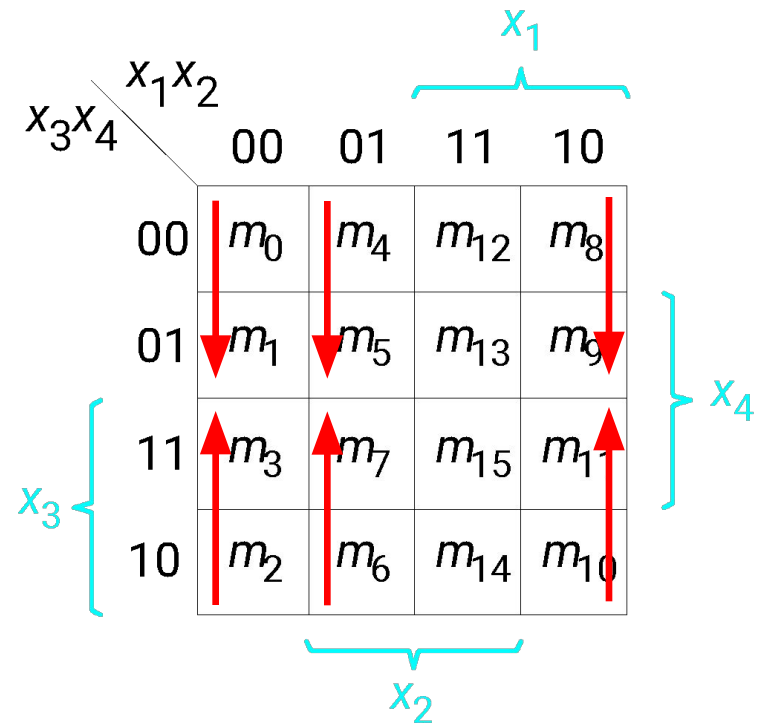
A four-variable Karnaugh map

x1	x2	x3	x4	
0	0	0	0	m0
0	0	0	1	m1
0	0	1	0	m2
0	0	1	1	m3
0	1	0	0	m4
0	1	0	1	m5
0	1	1	0	m6
0	1	1	1	m7
1	0	0	0	m8
1	0	0	1	m9
1	0	1	0	m10
1	0	1	1	m11
1	1	0	0	m12
1	1	0	1	m13
1	1	1	0	m14
1	1	1	1	m15



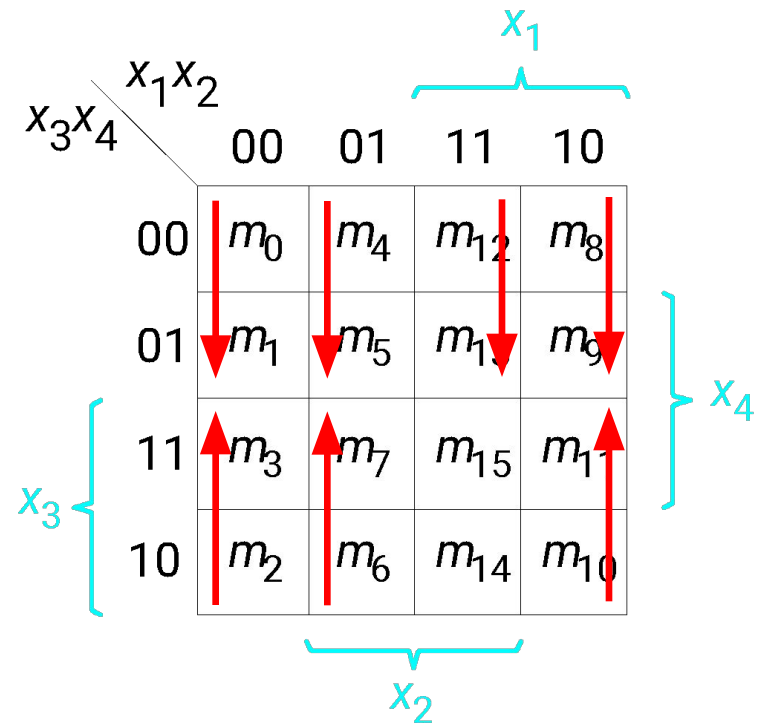
A four-variable Karnaugh map

x1	x2	x3	x4	
0	0	0	0	m0
0	0	0	1	m1
0	0	1	0	m2
0	0	1	1	m3
0	1	0	0	m4
0	1	0	1	m5
0	1	1	0	m6
0	1	1	1	m7
1	0	0	0	m8
1	0	0	1	m9
1	0	1	0	m10
1	0	1	1	m11
1	1	0	0	m12
1	1	0	1	m13
1	1	1	0	m14
1	1	1	1	m15



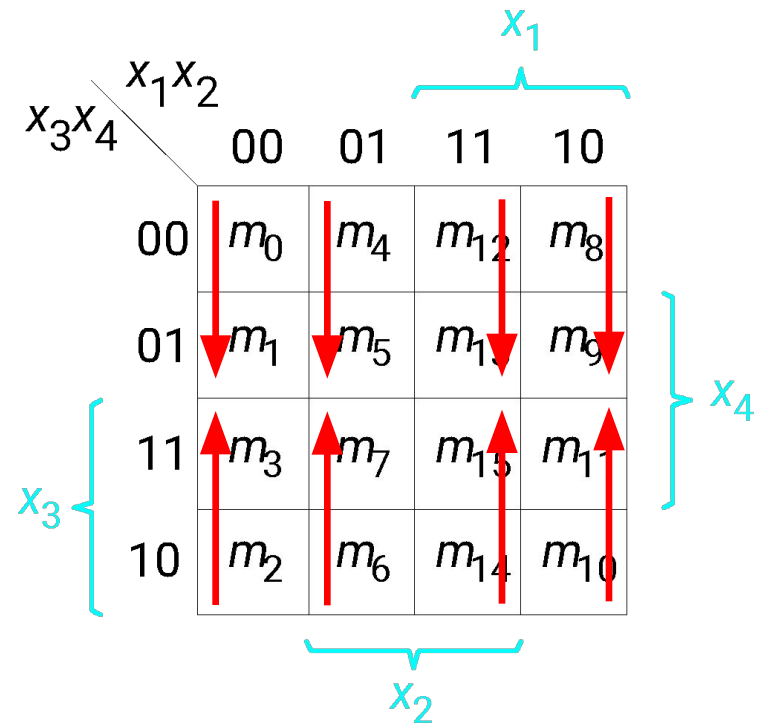
A four-variable Karnaugh map

x1	x2	x3	x4	
0	0	0	0	m0
0	0	0	1	m1
0	0	1	0	m2
0	0	1	1	m3
0	1	0	0	m4
0	1	0	1	m5
0	1	1	0	m6
0	1	1	1	m7
1	0	0	0	m8
1	0	0	1	m9
1	0	1	0	m10
1	0	1	1	m11
1	1	0	0	m12
1	1	0	1	m13
1	1	1	0	m14
1	1	1	1	m15

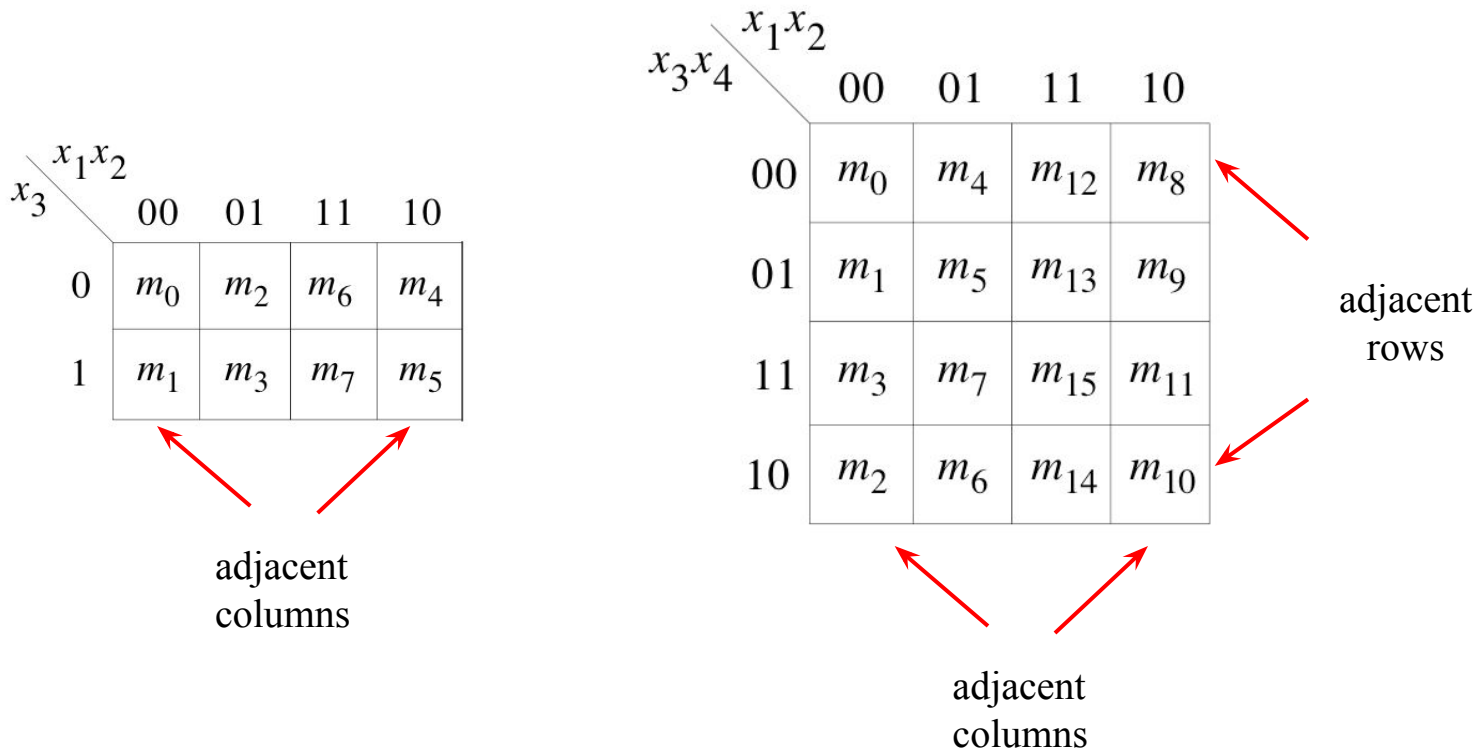


A four-variable Karnaugh map

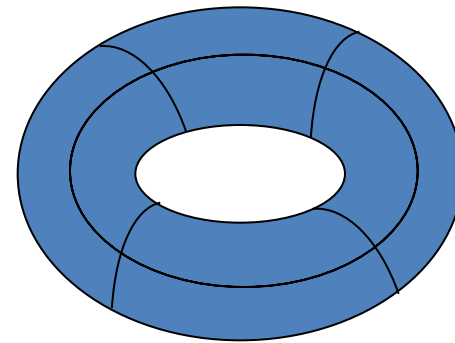
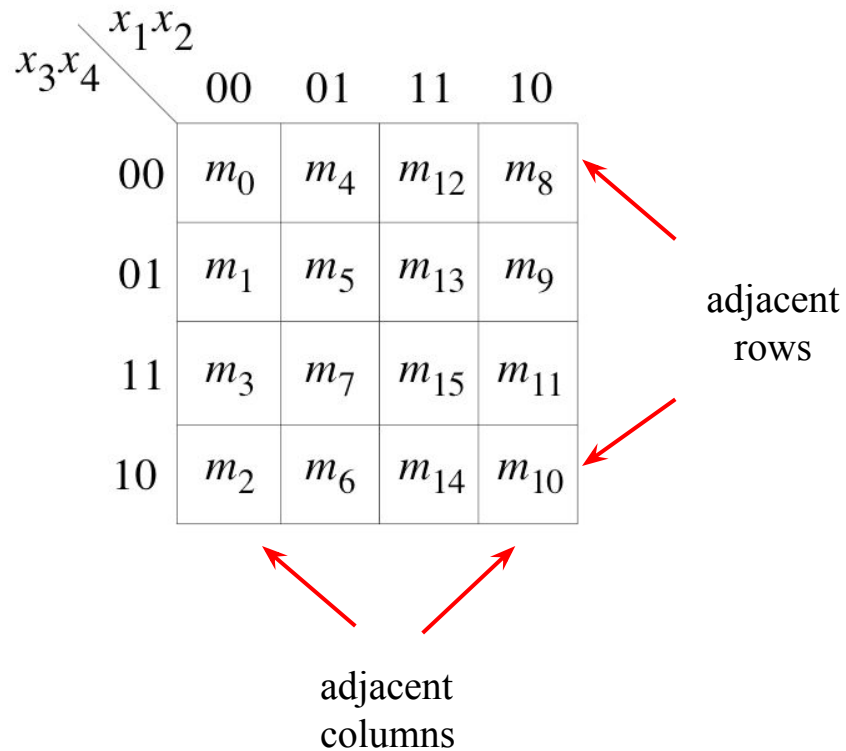
x1	x2	x3	x4	
0	0	0	0	m0
0	0	0	1	m1
0	0	1	0	m2
0	0	1	1	m3
0	1	0	0	m4
0	1	0	1	m5
0	1	1	0	m6
0	1	1	1	m7
1	0	0	0	m8
1	0	0	1	m9
1	0	1	0	m10
1	0	1	1	m11
1	1	0	0	m12
1	1	0	1	m13
1	1	1	0	m14
1	1	1	1	m15



Adjacency Rules



Adjacency Rules



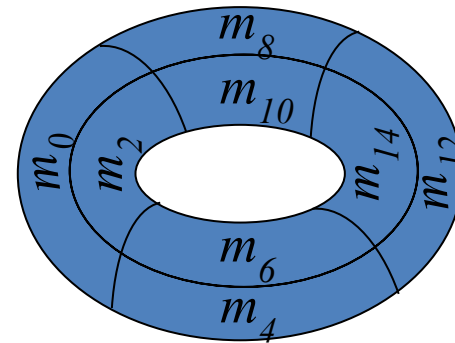
As if the K-map were drawn on a torus

Adjacency Rules

$x_3x_4 \backslash x_1x_2$		x_1x_2			
		00	01	11	10
x_3x_4	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

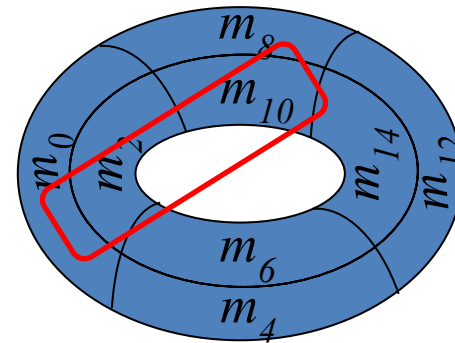
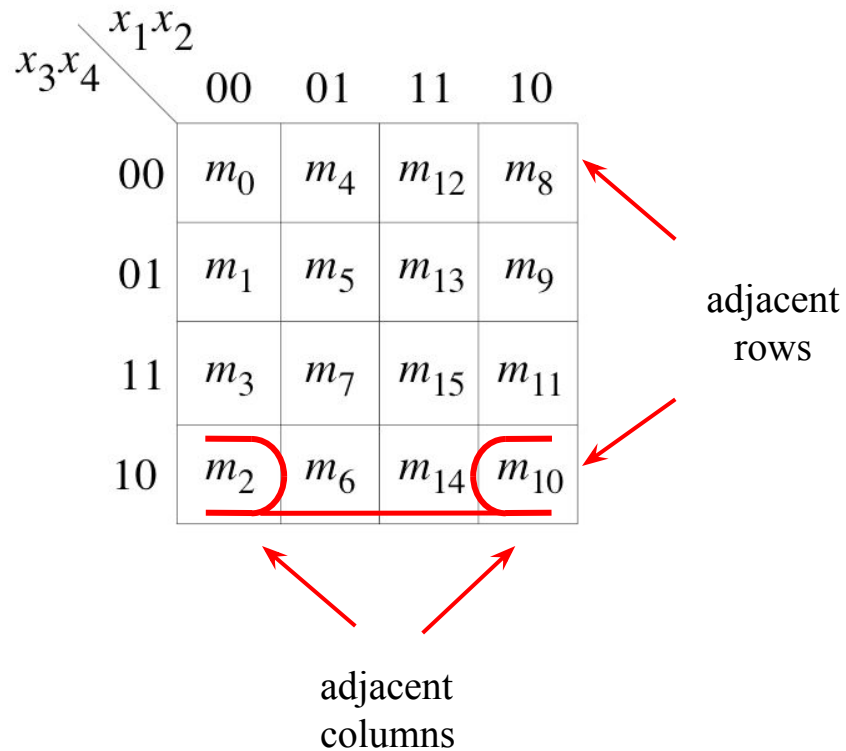
adjacent
rows

adjacent
columns



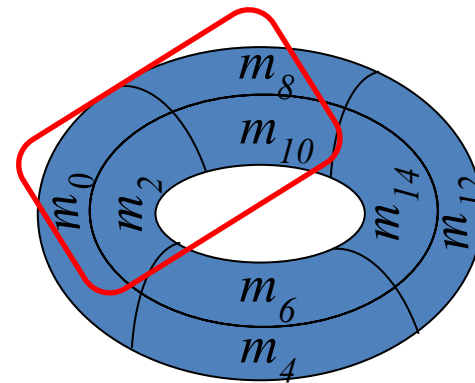
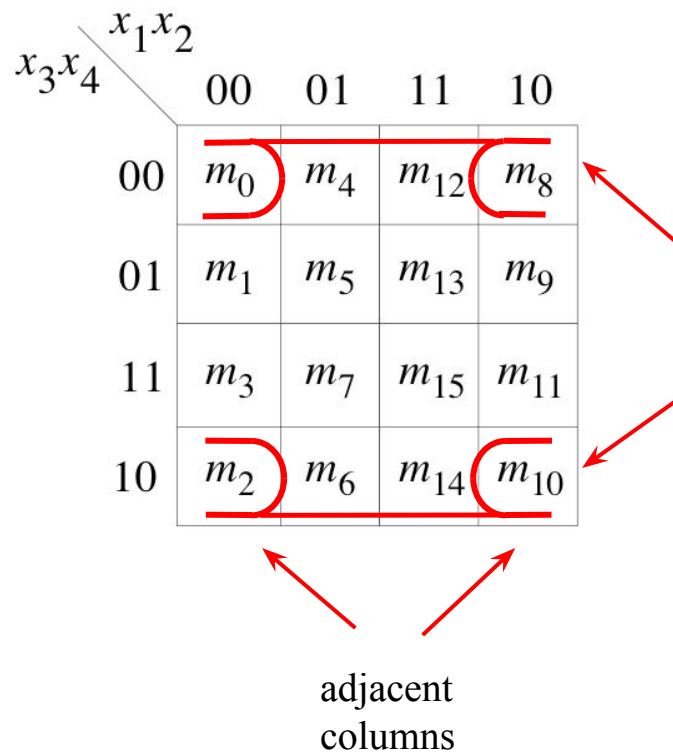
As if the K-map were
drawn on a torus

Adjacency Rules



As if the K-map were drawn on a torus

Adjacency Rules



As if the K-map were drawn on a torus

Some Valid Groupings

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

Groupings: x_1 (columns 11, 10), x_2 (columns 00, 01), x_3 (rows 11, 10), x_4 (rows 01, 11).

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

Groupings: x_1 (columns 11, 10), x_2 (columns 00, 01), x_3 (rows 11, 10), x_4 (rows 01, 11).

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

Groupings: x_1 (columns 11, 10), x_2 (columns 00, 01), x_3 (rows 11, 10), x_4 (rows 01, 11).

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

Groupings: x_1 (columns 11, 10), x_2 (columns 00, 01), x_3 (rows 11, 10), x_4 (rows 01, 11).

Some Valid Groupings

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}
		x_2			

Groupings: x_1 (columns 11, 10), x_2 (columns 00, 01), x_3 (rows 11, 10), x_4 (rows 01, 11).

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}
		x_2			

Groupings: x_1 (columns 11, 10), x_2 (columns 00, 01), x_3 (rows 11, 10), x_4 (rows 01, 11).

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}
		x_2			

Groupings: x_1 (columns 11, 10), x_2 (columns 00, 01), x_3 (rows 11, 10), x_4 (rows 01, 11).

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}
		x_2			

Groupings: x_1 (columns 11, 10), x_2 (columns 00, 01), x_3 (rows 11, 10), x_4 (rows 01, 11).

Some Valid Groupings

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}
		x_2			

Groupings: A red box highlights the 2x2 subgrid containing m_0, m_4, m_1, m_5 . A blue bracket on the left indicates the x_3 dimension, and a blue bracket on the right indicates the x_4 dimension.

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}
		x_2			

Groupings: A red box highlights the 4x1 subgrid containing m_8, m_9, m_{11}, m_{10} . A blue bracket on the left indicates the x_3 dimension, and a blue bracket on the right indicates the x_4 dimension.

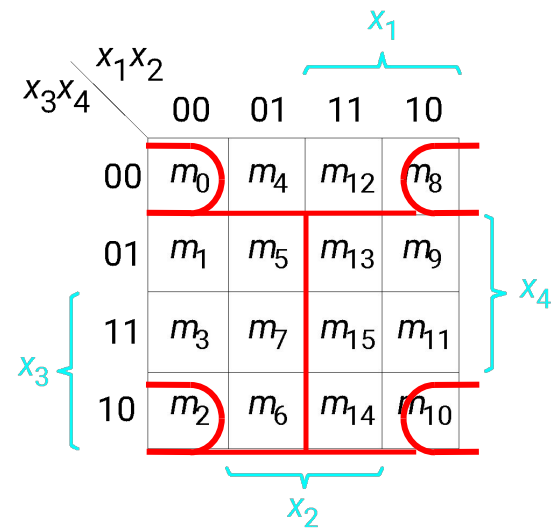
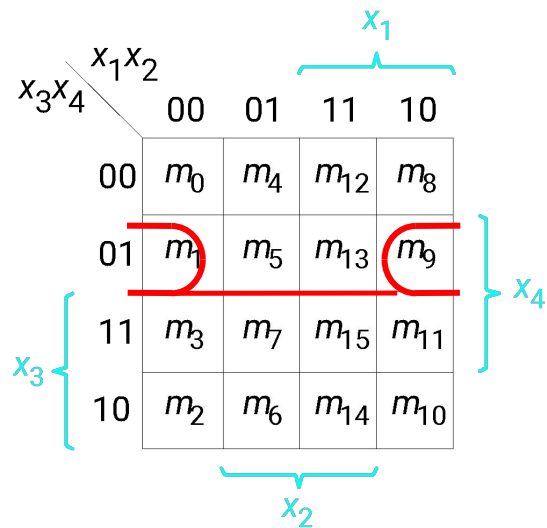
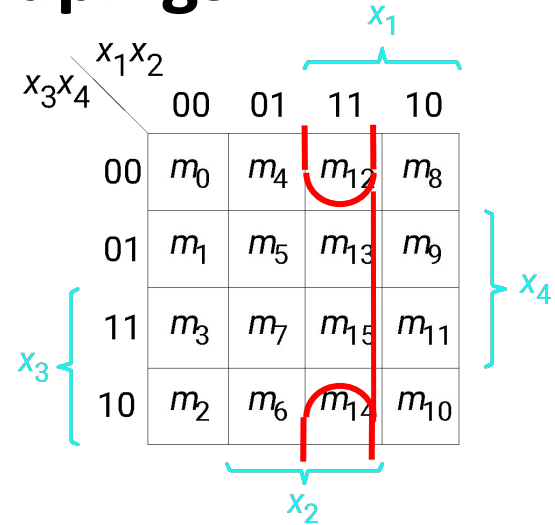
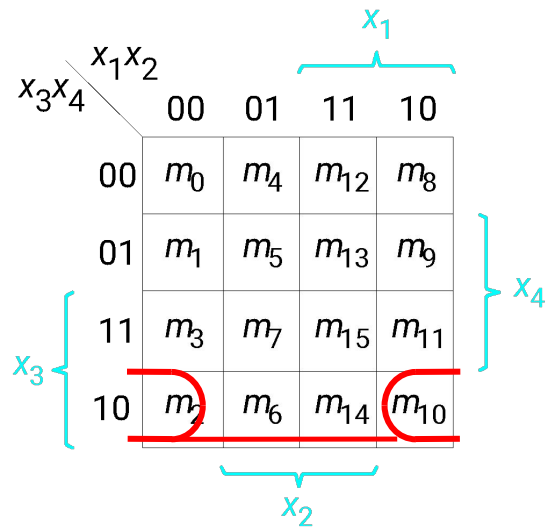
$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}
		x_2			

Groupings: A red box highlights the 2x2 subgrid containing m_5, m_{13}, m_7, m_{15} . A blue bracket on the left indicates the x_3 dimension, and a blue bracket on the right indicates the x_4 dimension.

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}
		x_2			

Groupings: A red box highlights the 2x2 subgrid containing m_1, m_5, m_3, m_7 . A blue bracket on the left indicates the x_3 dimension, and a blue bracket on the right indicates the x_4 dimension.

Some Valid Groupings



Some Invalid Groupings

	x_1x_2		x_1		
x_3x_4	00	01	11	10	
00	m_0	m_4	m_{12}	m_8	
01	m_1	m_5	m_{13}	m_9	
11	m_3	m_7	m_{15}	m_{11}	
10	m_2	m_6	m_{14}	m_{10}	
			x_2		
	x_3				x_4

	x_1x_2		x_1		
x_3x_4	00	01	11	10	
00	m_0	m_4	m_{12}	m_8	
01	m_1	m_5	m_{13}	m_9	
11	m_3	m_7	m_{15}	m_{11}	
10	m_2	m_6	m_{14}	m_{10}	
			x_2		
	x_3				x_4

All sides must be powers of 2.

	x_1x_2		x_1		
x_3x_4	00	01	11	10	
00	m_0	m_4	m_{12}	m_8	
01	m_1	m_5	m_{13}	m_9	
11	m_3	m_7	m_{15}	m_{11}	
10	m_2	m_6	m_{14}	m_{10}	
			x_2		
	x_3				x_4

	x_1x_2		x_1		
x_3x_4	00	01	11	10	
00	m_0	m_4	m_{12}	m_8	
01	m_1	m_5	m_{13}	m_9	
11	m_3	m_7	m_{15}	m_{11}	
10	m_2	m_6	m_{14}	m_{10}	
			x_2		
	x_3				x_4

Some **valid** Groupings

A 4x4 Karnaugh map for variables x_1, x_2, x_3, x_4 . The columns are labeled x_1x_2 (00, 01, 11, 10) and the rows are labeled x_3x_4 (00, 01, 11, 10). The cells contain minterms m_0 through m_{15} . Groupings are shown with colored brackets: a red bracket for x_1 (columns 01 and 11), a blue bracket for x_2 (columns 00 and 01), a green bracket for x_3 (rows 00 and 01), and a purple bracket for x_4 (rows 01 and 11).

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00	m_0	m_4	m_{12}	m_8
01	m_1	m_5	m_{13}	m_9
11	m_3	m_7	m_{15}	m_{11}
10	m_2	m_6	m_{14}	m_{10}

A 4x4 Karnaugh map for variables x_1, x_2, x_3, x_4 . The columns are labeled x_1x_2 (00, 01, 11, 10) and the rows are labeled x_3x_4 (00, 01, 11, 10). The cells contain minterms m_0 through m_{15} . Groupings are shown with colored brackets: a red bracket for x_1 (columns 11 and 10), a blue bracket for x_2 (columns 01 and 11), a green bracket for x_3 (rows 01 and 11), and a purple bracket for x_4 (rows 00 and 01).

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00	m_0	m_4	m_{12}	m_8
01	m_1	m_5	m_{13}	m_9
11	m_3	m_7	m_{15}	m_{11}
10	m_2	m_6	m_{14}	m_{10}

All sides must be powers of 2.

A 4x4 Karnaugh map for variables x_1, x_2, x_3, x_4 . The columns are labeled x_1x_2 (00, 01, 11, 10) and the rows are labeled x_3x_4 (00, 01, 11, 10). The cells contain minterms m_0 through m_{15} . Groupings are shown with colored brackets: a red bracket for x_1 (columns 01 and 11), a blue bracket for x_2 (columns 01 and 11), a green bracket for x_3 (rows 01 and 11), and a purple bracket for x_4 (rows 01 and 11).

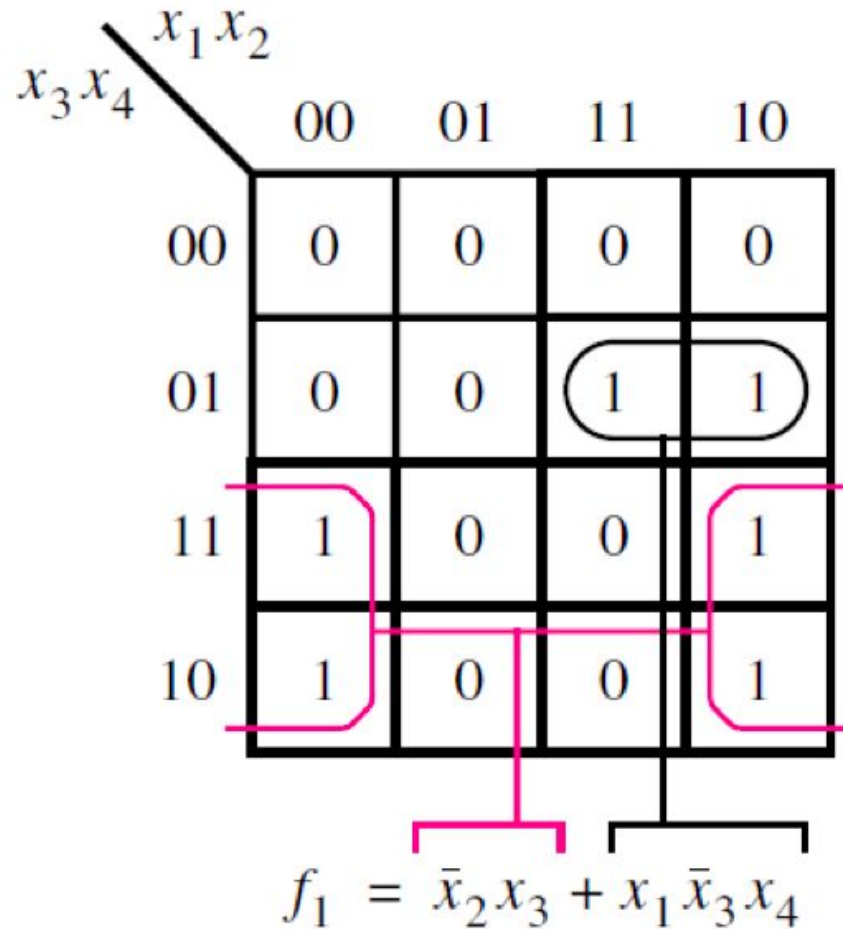
$x_3x_4 \backslash x_1x_2$	00	01	11	10
00	m_0	m_4	m_{12}	m_8
01	m_1	m_5	m_{13}	m_9
11	m_3	m_7	m_{15}	m_{11}
10	m_2	m_6	m_{14}	m_{10}

A 4x4 Karnaugh map for variables x_1, x_2, x_3, x_4 . The columns are labeled x_1x_2 (00, 01, 11, 10) and the rows are labeled x_3x_4 (00, 01, 11, 10). The cells contain minterms m_0 through m_{15} . Groupings are shown with colored brackets: a red bracket for x_1 (columns 01 and 11), a blue bracket for x_2 (columns 01 and 11), a green bracket for x_3 (rows 01 and 11), and a purple bracket for x_4 (rows 01 and 11).

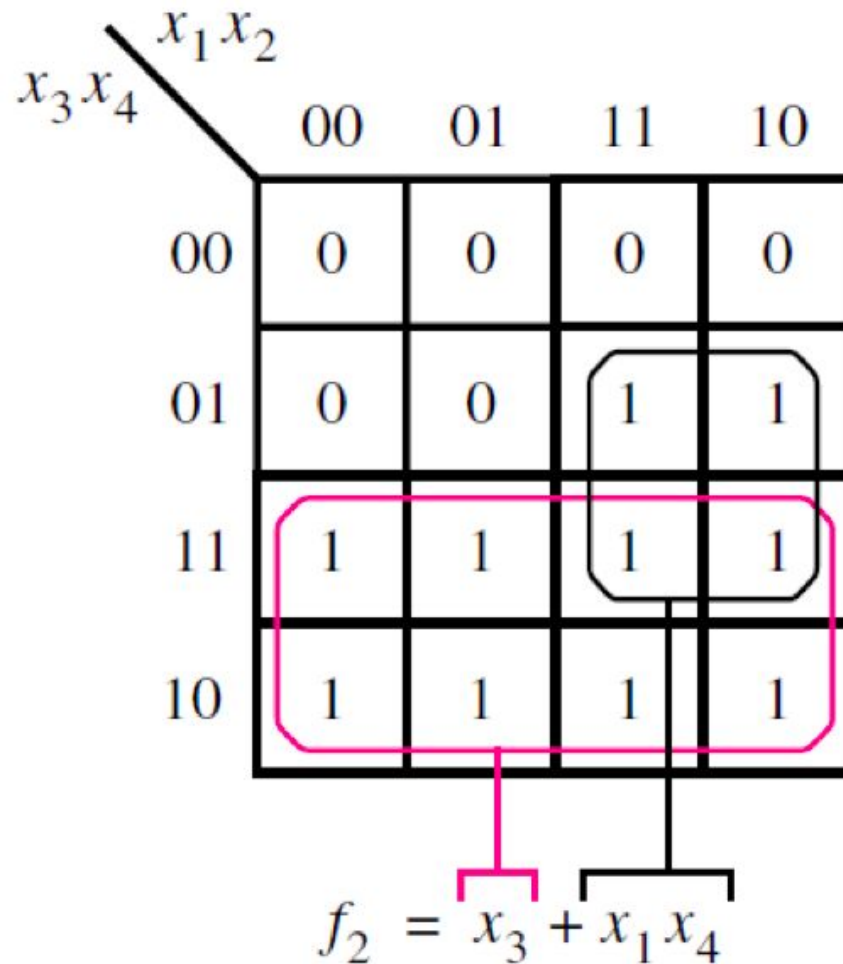
$x_3x_4 \backslash x_1x_2$	00	01	11	10
00	m_0	m_4	m_{12}	m_8
01	m_1	m_5	m_{13}	m_9
11	m_3	m_7	m_{15}	m_{11}
10	m_2	m_6	m_{14}	m_{10}

Minimization Examples with 4-variable K-Maps

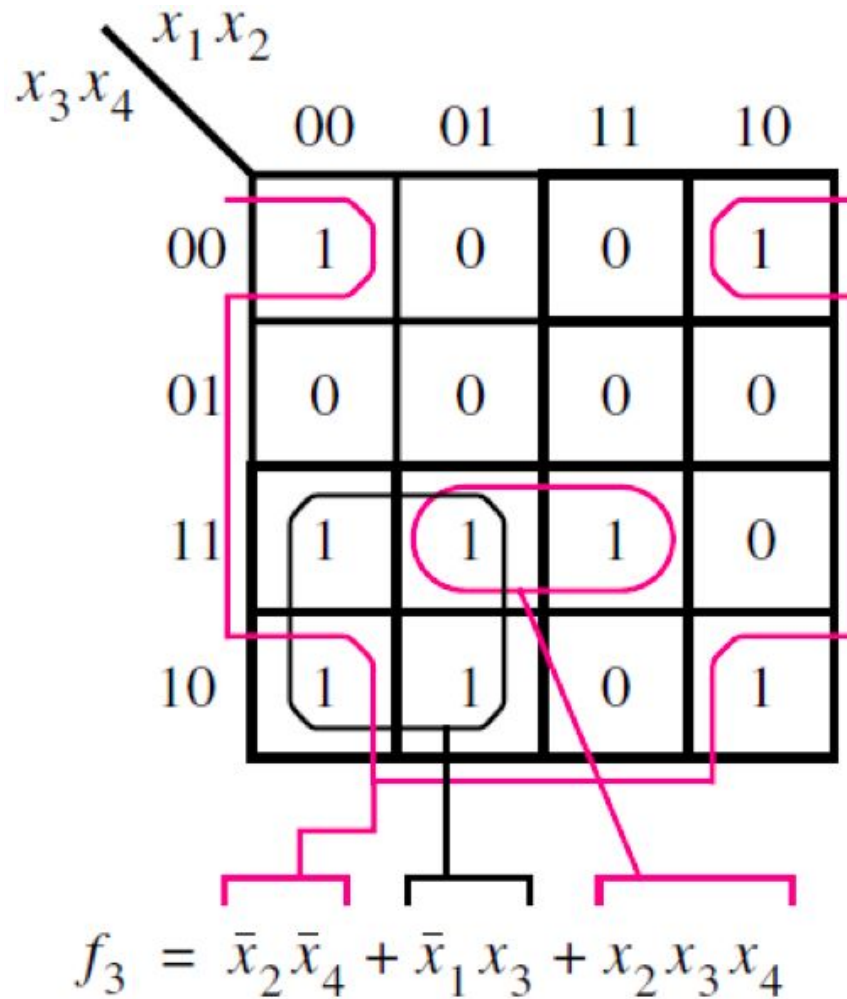
Example of a four-variable Karnaugh map



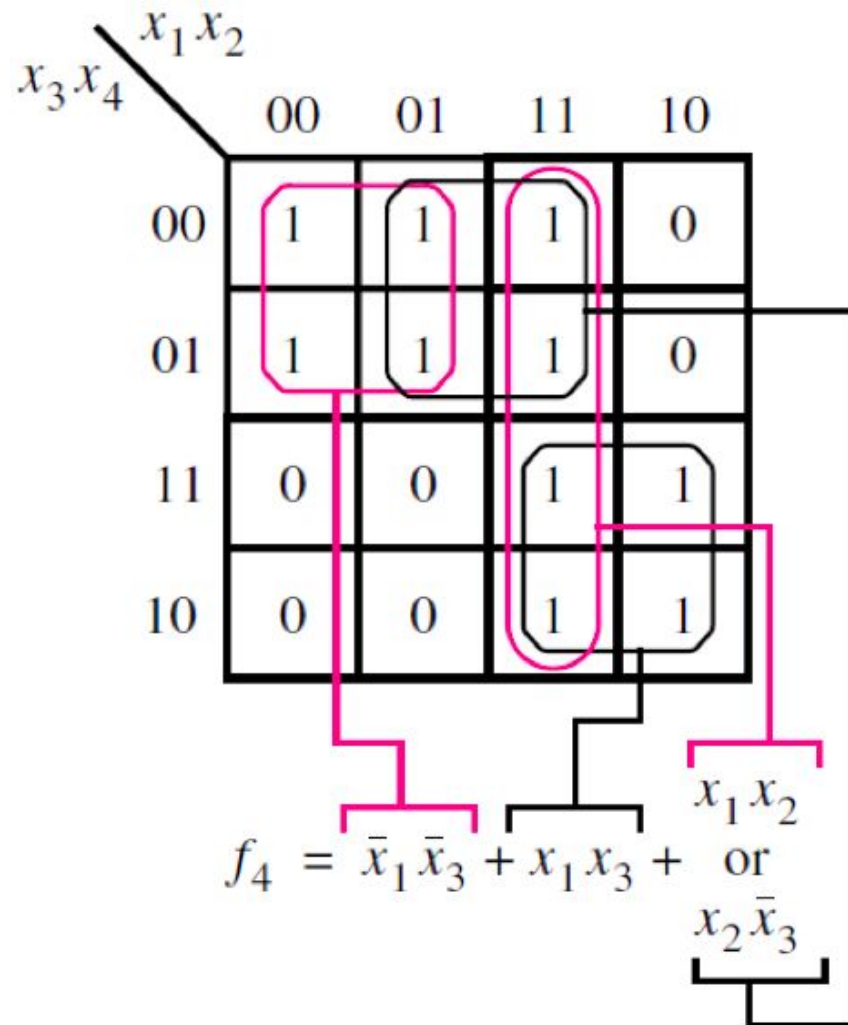
Example of a four-variable Karnaugh map



Example of a four-variable Karnaugh map

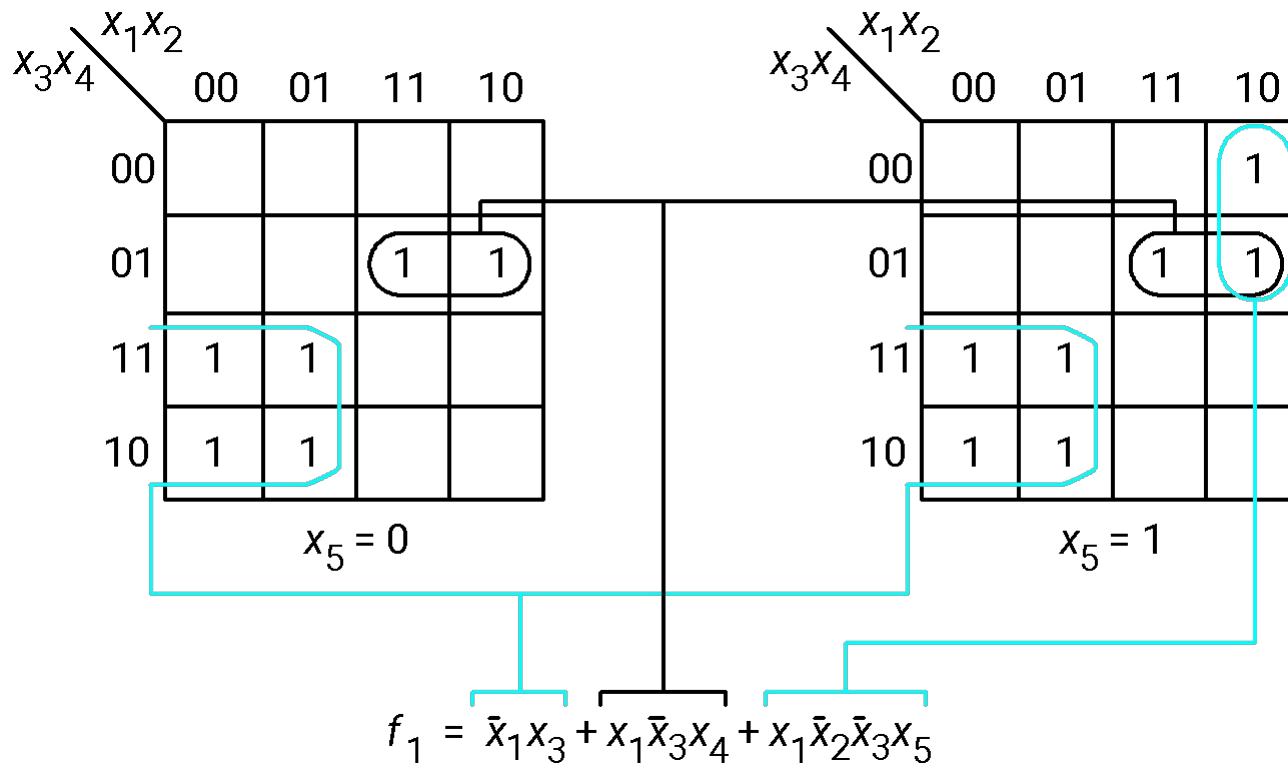


Example of a four-variable Karnaugh map



Five-Variable K-Map

A five-variable Karnaugh map



Don't Care Conditions

- There may be a combination of input values which
 - will **never** occur
 - if they do occur, the output is of no concern.
- The function value for such combinations is called a *don't care*.
- They are denoted with **x** or **—**. Each **x** may be arbitrarily assigned the value 0 or 1 in an implementation.
- Don't cares can be used to **further** simplify a function

Don't-Care Conditions

- Some logic circuits can be designed so that there are certain input conditions for which there are no specified output levels.
- A circuit designer is free to make the output for any don't care condition either a 0 or a 1 in order to produce the simplest output expression.

Example

- Simplify the function $f(a,b,c,d)$ whose K-map is shown at the right.
- $f = a'c'd + ab' + cd' + a'bc'$
or
- $f = a'c'd + ab' + cd' + a'bd'$

ab \ cd				
	00	01	11	10
00	0	1	0	1
01	1	1	0	1
11	0	0	x	x
10	1	1	x	x

0	1	0	1
1	1	0	1
0	0	x	x
1	1	x	x

0	1	0	1
1	1	0	1
0	0	x	x
1	1	x	x

Another Example

- Simplify the function $g(a,b,c,d)$ whose K-map is shown at right.
- $g = a'c' + ab$
or
- $g = a'c' + b'd$

ab \ cd					
		cd			
		x	1	0	0
		1	x	0	x
		1	x	x	1
		0	x	x	0

		x	1	0	0
		1	x	0	x
		1	x	x	1
		0	x	x	0

		x	1	0	0
		1	x	0	x
		1	x	x	1
		0	x	x	0

Another Example

For each symbol of the Excess-3 code, we use 1's to draw the map for simplifying Boolean function.

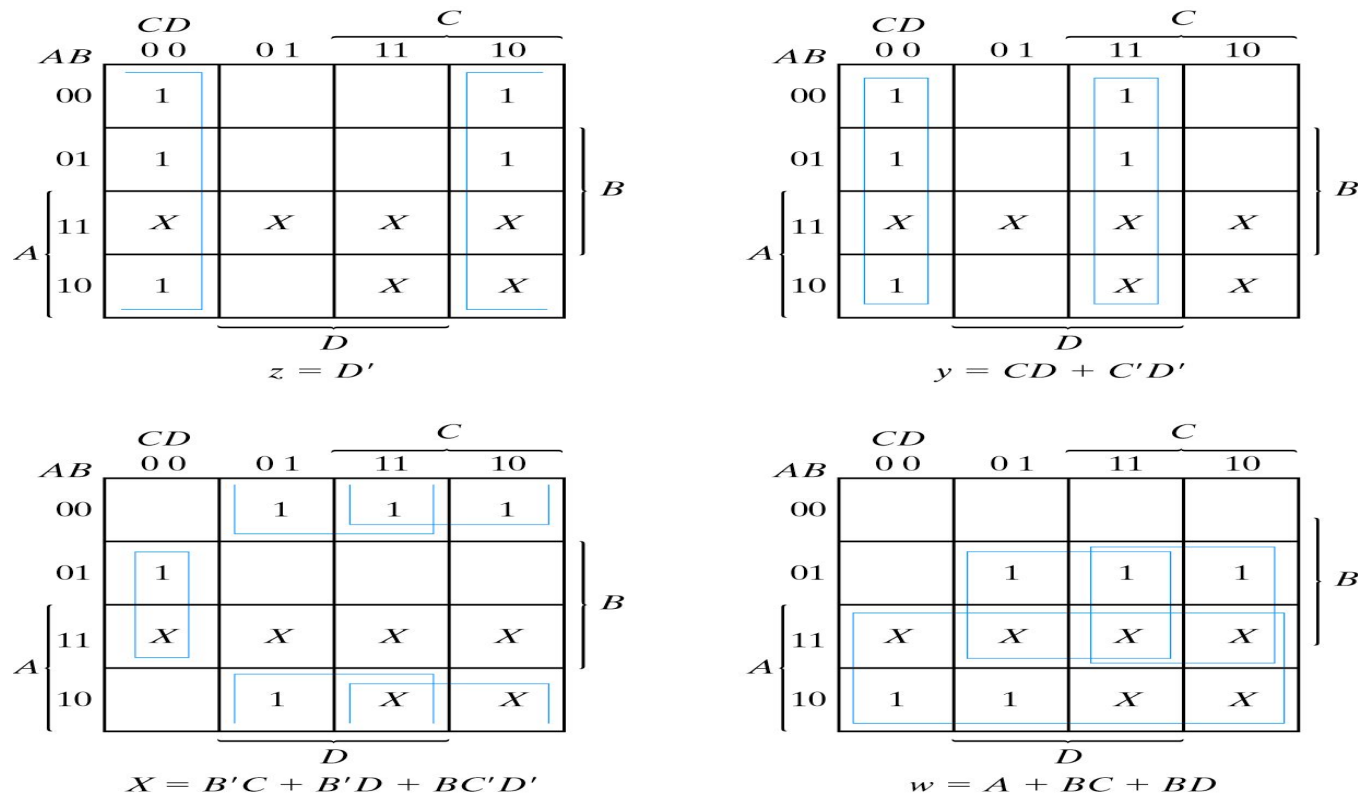


Fig. 4-3 Maps for BCD to Excess-3 Code Converter

Circuit implementation

$$z = D'; \quad y = CD + C'D' = CD + (C + D)'$$

$$x = B'C + B'D + BC'D' = B'(C + D) + B(C + D)'$$

$$w = A + BC + BD = A + B(C + D)$$

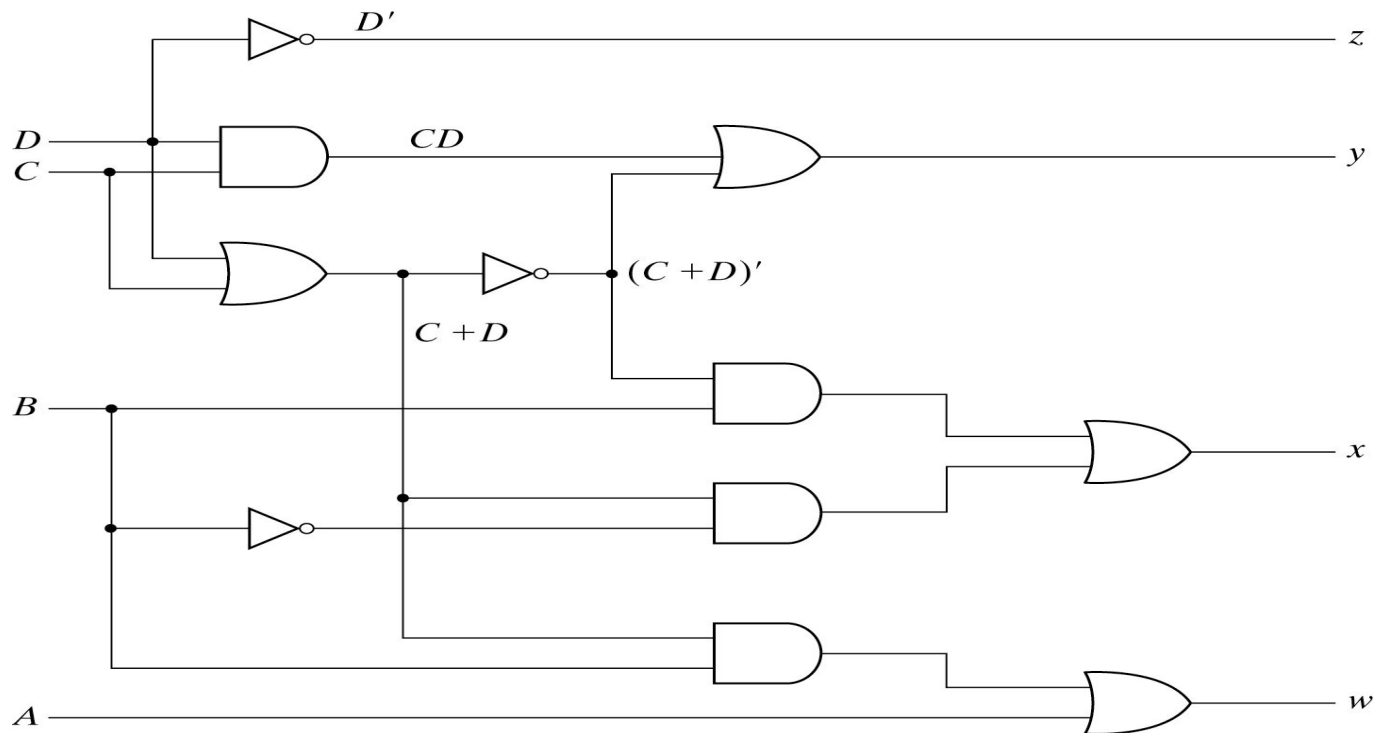
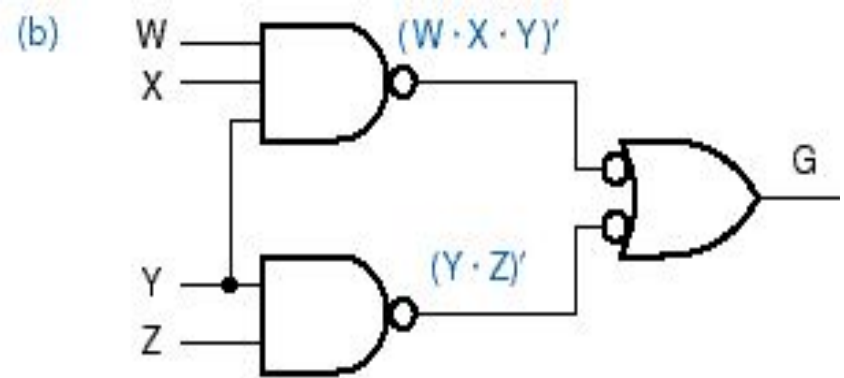
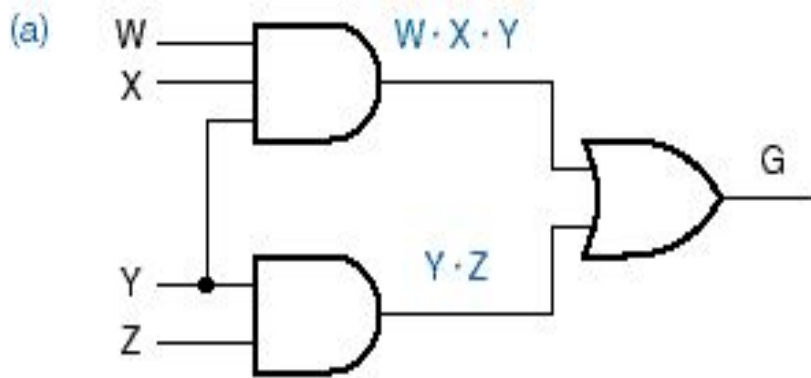


Fig. 4-4 Logic Diagram for BCD to Excess-3 Code Converter

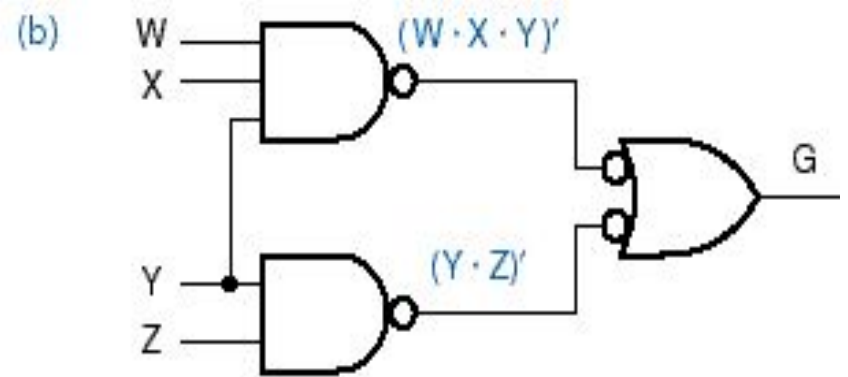
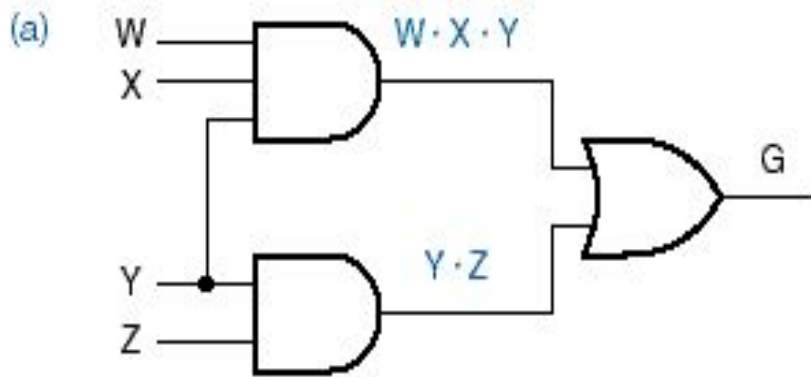
AND-OR (SOP) Emulation Using NANDs



Two-level implementations

- a) Original SOP
- b) Implementation with NANDs

AND-OR (SOP) Emulation Using NANDs (cont.)

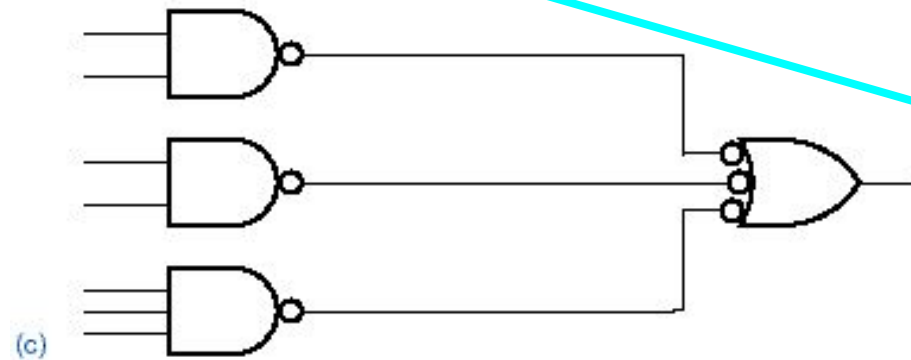
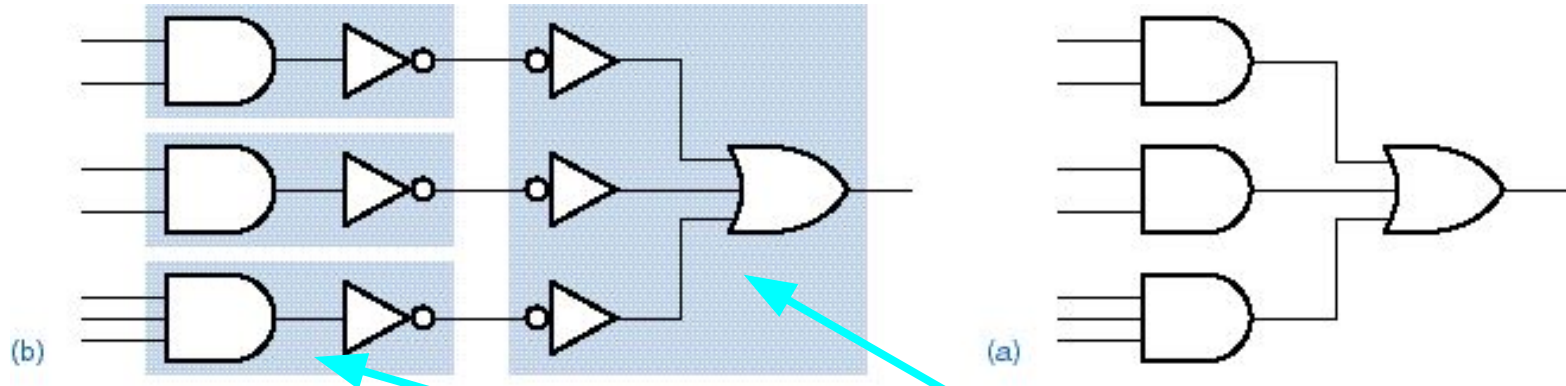


Verify:

(a) $G = WXY + YZ$

(b) $G = ((WXY)' \cdot (YZ)')'$
 $= (WXY)'' + (YZ)'' = WXY + YZ$

SOP with NAND



- (a) Original SOP
- (b) Double inversion and grouping
- (c) Replacement with NANDs

AND-NOT

NOT-OR

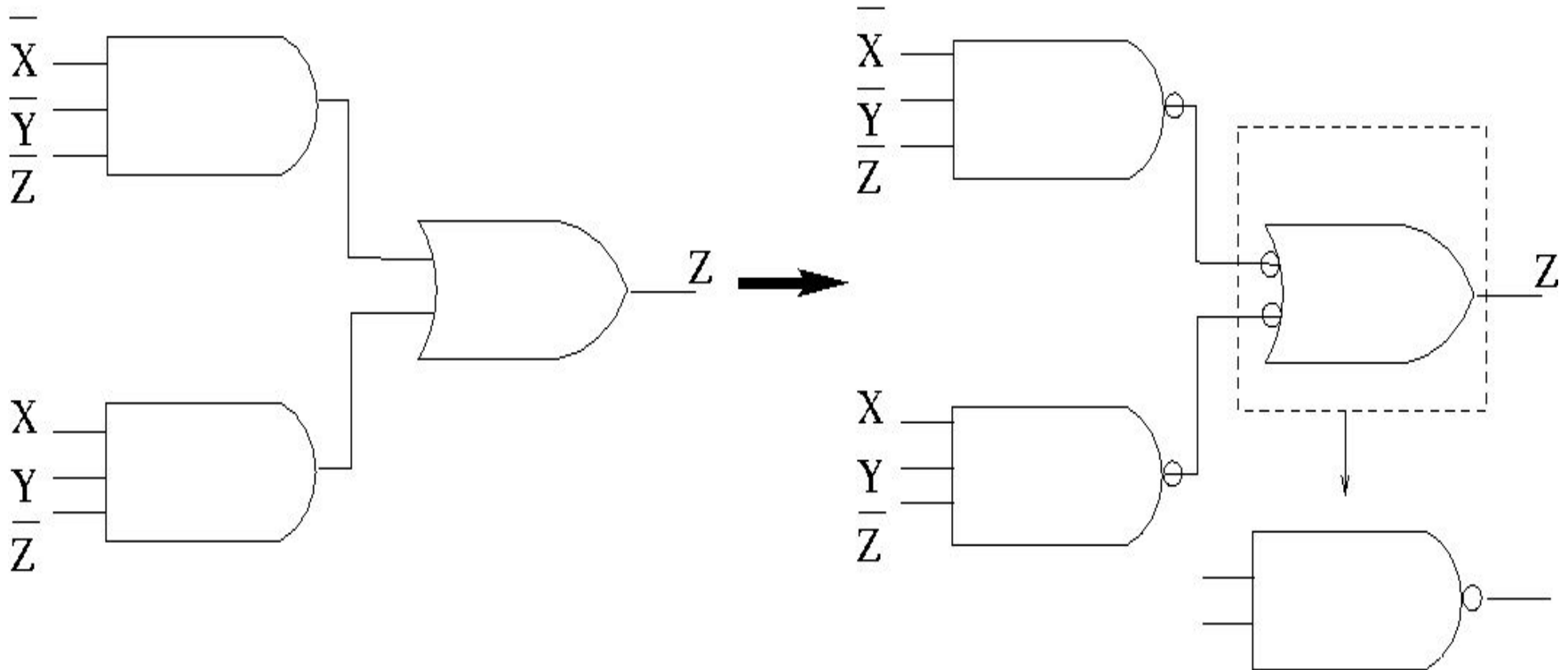
Copyright © 2000 by Prentice Hall, Inc.
Digital Design Principles and Practices, 3/e

Two-Level NAND Gate Implementation - Example

$$F(X,Y,Z) = \Sigma m(0,6)$$

1. Express F in SOP form:
$$F = X'Y'Z' + XYZ'$$
2. Obtain the AND-OR implementation for F .
3. Add bubbles and inverters to transform AND-OR to NAND-NAND gates.

Example (cont.)



Two-level implementation with NANDs

$$F = X'Y'Z' + XYZ'$$

Multilevel NAND Circuits

Starting from a multilevel circuit:

1. Convert all AND gates to NAND gates with AND-NOT graphic symbols.
2. Convert all OR gates to NAND gates with NOT-OR graphic symbols.
3. Check all the bubbles in the diagram. For every bubble that is not counteracted by another bubble along the same line, insert a NOT gate or complement the input literal from its original appearance.

Example

Use NAND gates and NOT gates to implement
 $Z = E'F(AB + C' + D') + GH$

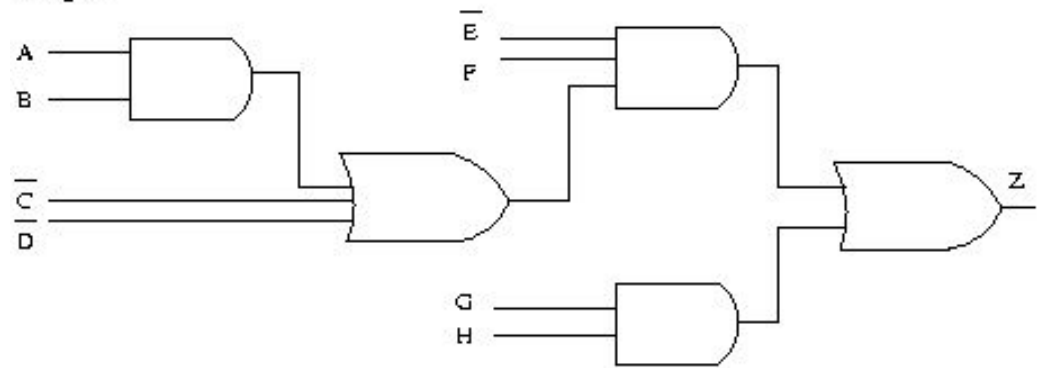
AB

$AB + C' + D'$

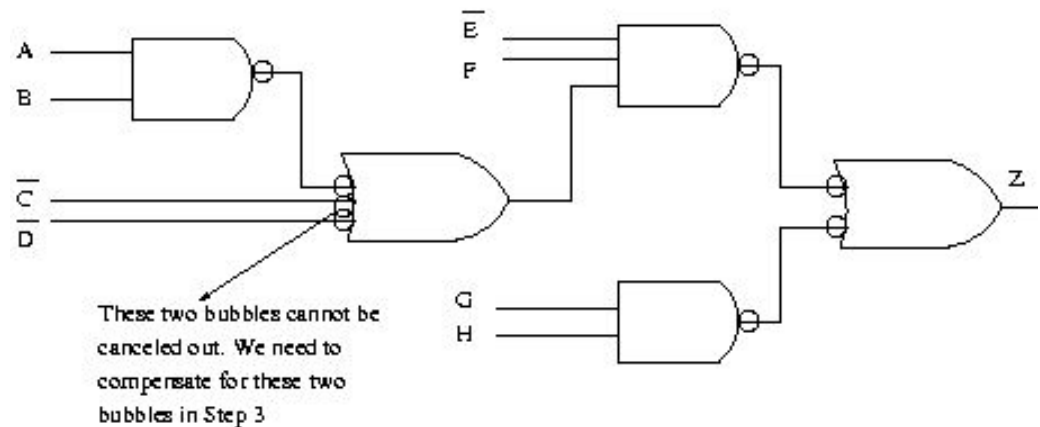
$E'F(AB + C' + D')$

$E'F(AB + C' + D') + GH$

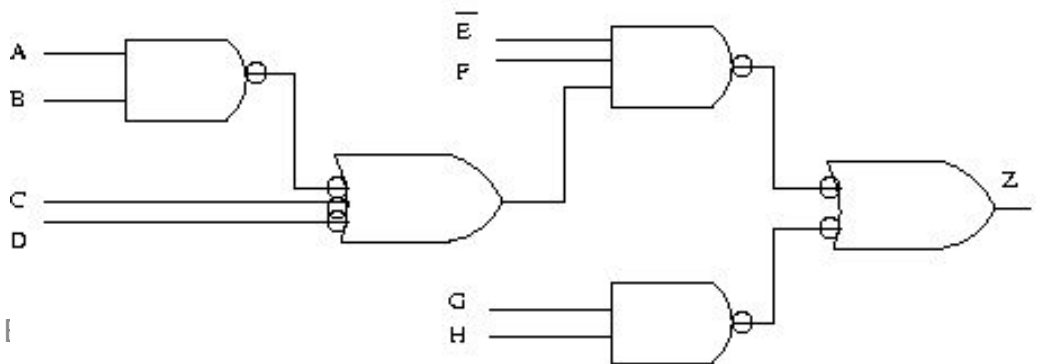
Step 1



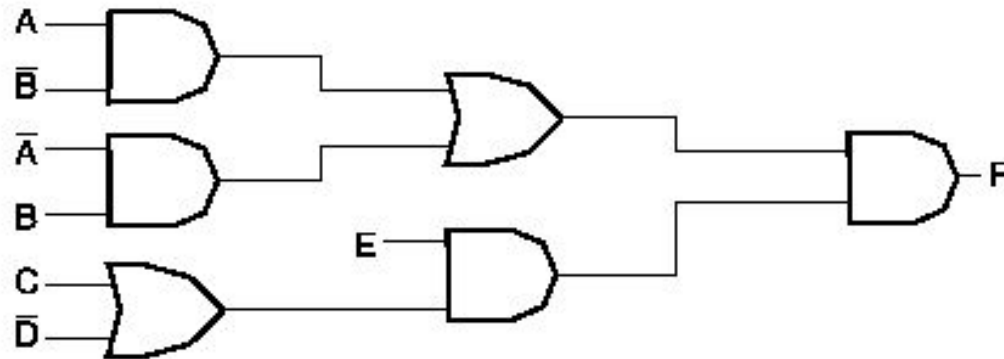
Step 2



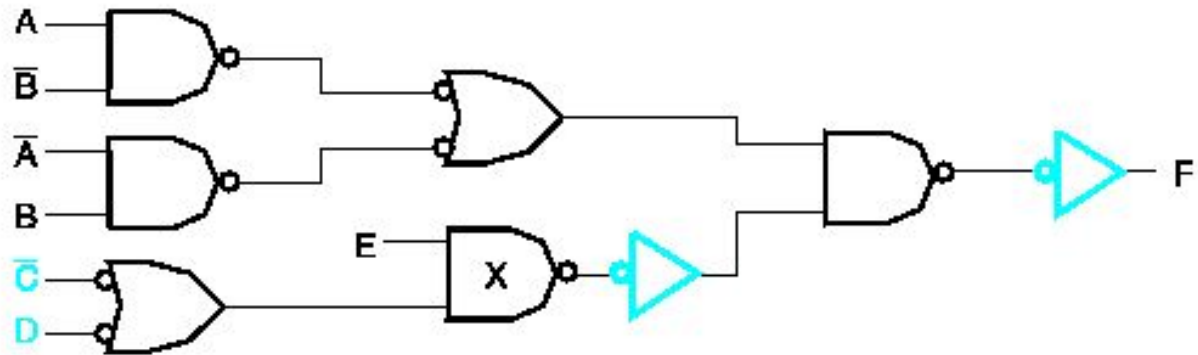
Step 3



Yet Another Example!



(a) AND – OR gates



(b) NAND gates

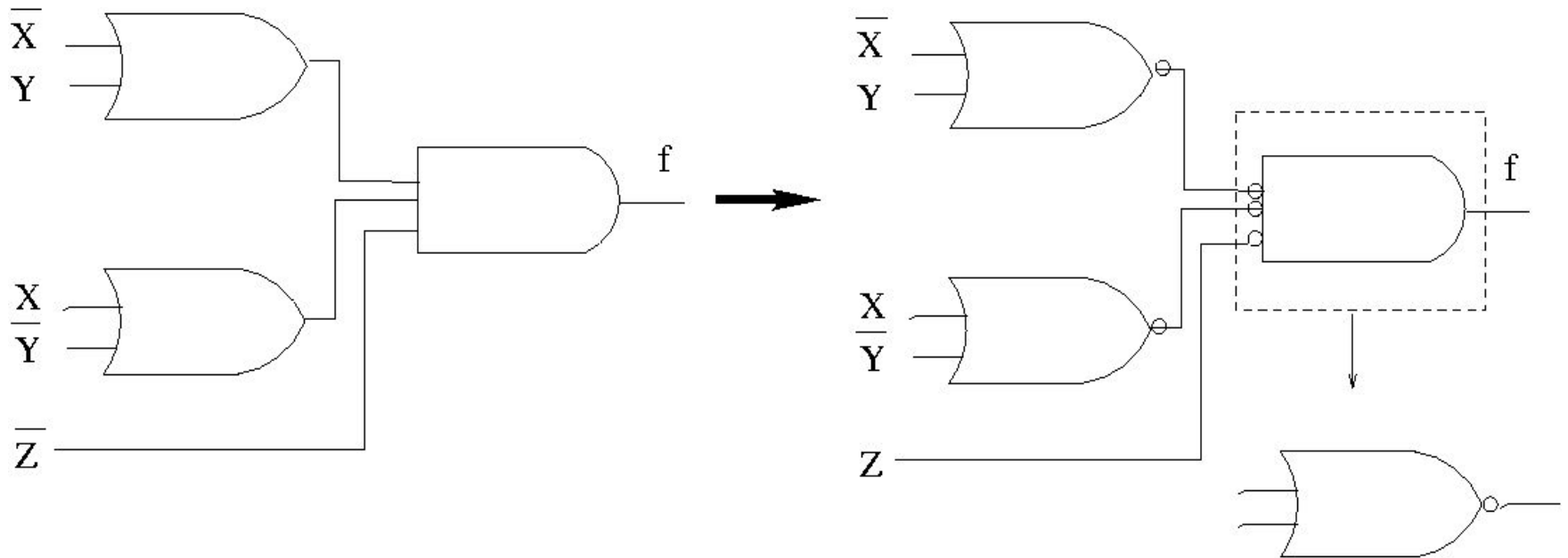
Fig. 2-32 Implementing $F = (A\bar{B} + \bar{A}B)E(C + \bar{D})$

Two-Level NOR Gate Implementation - Example

$$F(X,Y,Z) = \Sigma m(0,6)$$

1. Express F' in SOP form:
 1. $F' = \Sigma m(1,2,3,4,5,7)$
 $= X'Y'Z + X'YZ' + X'YZ + XY'Z' + XY'Z + XYZ$
 2. $F' = XY' + X'Y + Z$
2. Take the complement of F' to get F in the POS form: $F = (F')' = (X'+Y)(X+Y')Z'$
3. Obtain the OR-AND implementation for F .
4. Add bubbles and inverters to transform OR-AND implementation to NOR-NOR implementation.

Example (cont.)



Two-level implementation with NORs

$$F = (F')' = (X' + Y)(X + Y')Z'$$

Designing Combinational Logic Circuits (Procedure)

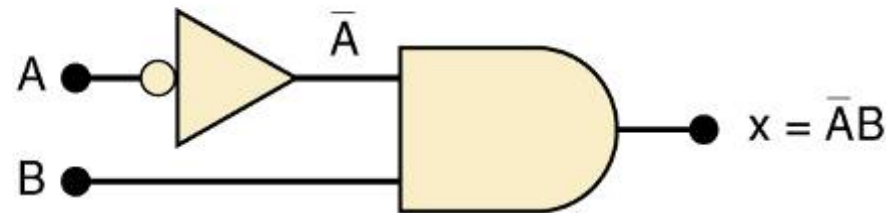
- To solve any logic design problem:
 1. Interpret the problem and set up its truth table.
 2. Write the AND (product) term for each case where output = 1.
 3. Combine the terms in SOP form.
 4. Simplify the output expression if possible.
 5. Implement the circuit for the final, simplified expression.

Designing Combinational Logic Circuits

Example - 1

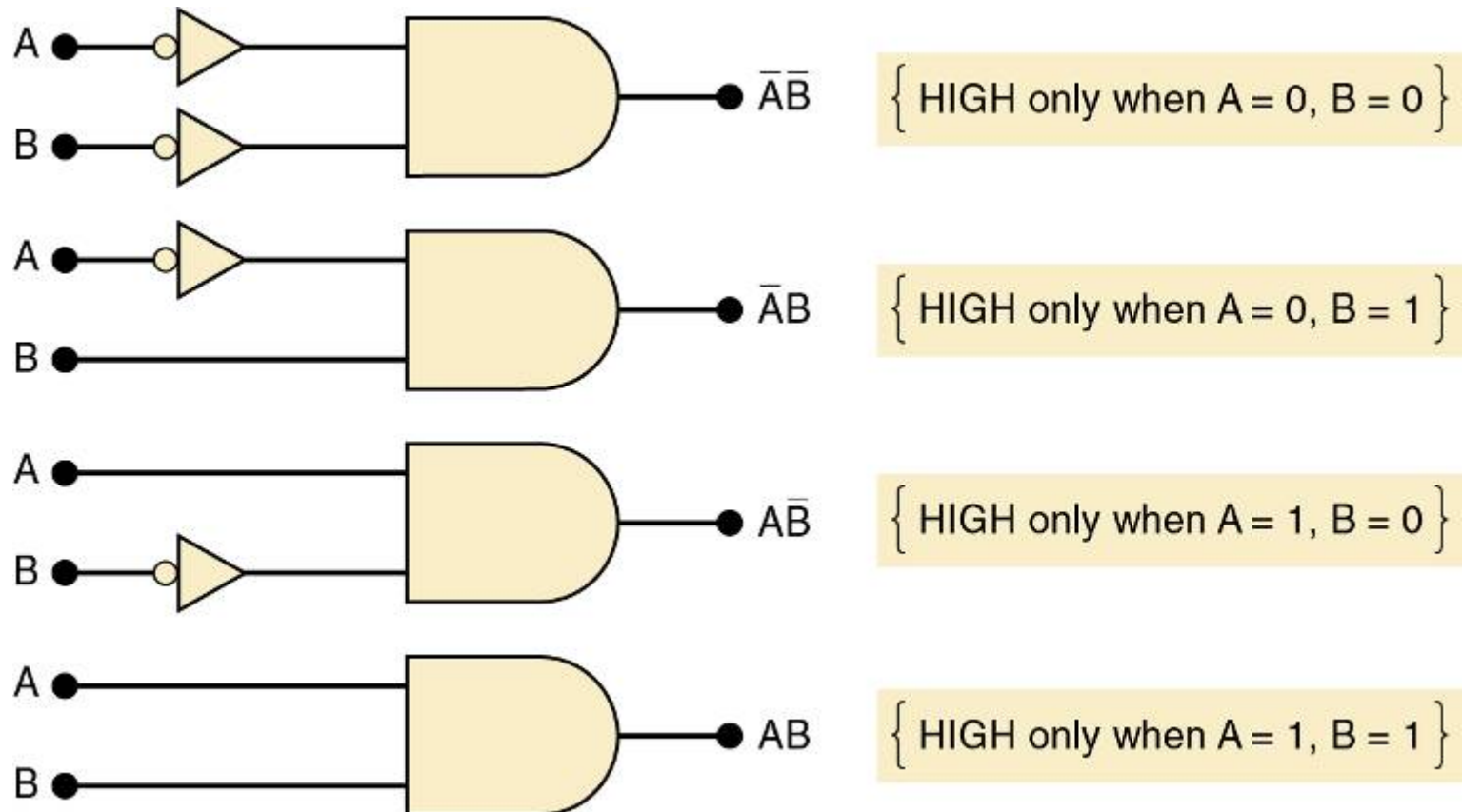
Circuit that produces a 1 output only for the $A = 0$, $B = 1$ condition.

A	B	x
0	0	0
0	1	1
1	0	0
1	1	0



Designing Combinational Logic Circuits

An **AND** gate with appropriate inputs can be used to produce a HIGH output for a specific set of input levels.

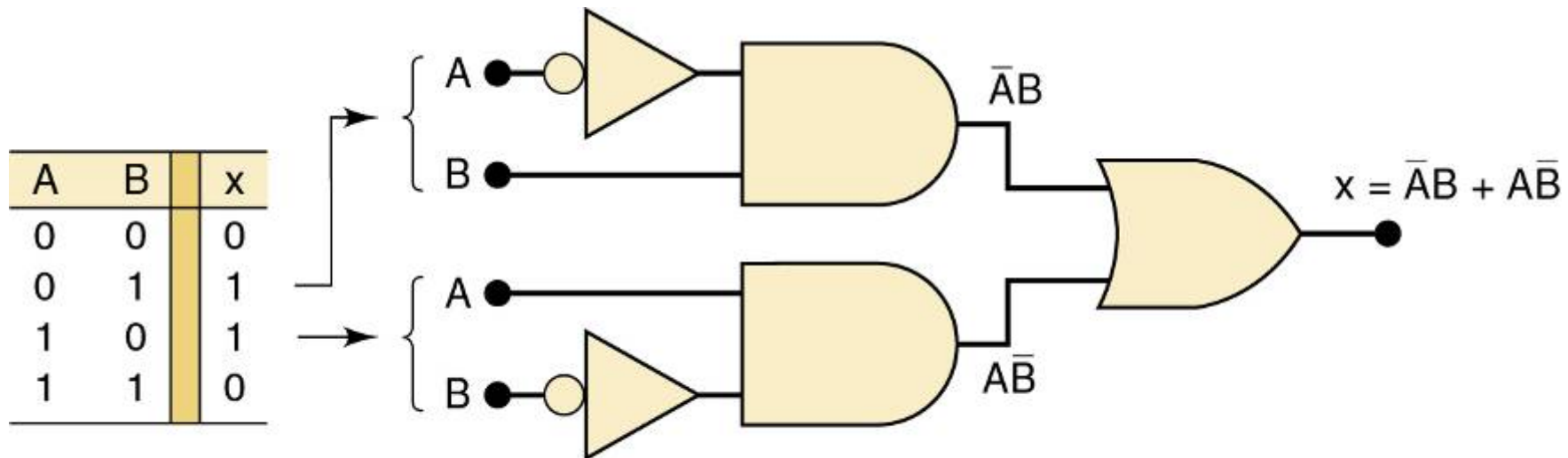


Designing Combinational Logic Circuits

Example - 2

Each set of input conditions that is to produce a 1 output is implemented by a separate **AND** gate.

The **AND** outputs are **OR**ed to produce the final output.



Designing Combinational Logic Circuits

Example - 3

Truth table for a 3-input circuit.

<i>A</i>	<i>B</i>	<i>C</i>	<i>x</i>
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

$\rightarrow \bar{A}\bar{B}\bar{C}$

$\rightarrow \bar{A}BC$

$\rightarrow ABC$

AND terms for each case where output is 1.

- Now, add these terms
- Simplify the expression if possible
- Draw the circuit Diagram

Designing Combinational Logic Circuits

Example - 4

Design a logic circuit with three inputs, A, B, and C. Output to be HIGH only when a majority inputs are HIGH.

Truth table.

A	B	C	x
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

AND terms for each case where output is 1.

$$\rightarrow \bar{A}BC$$

$$\rightarrow A\bar{B}C$$

$$\rightarrow AB\bar{C}$$

$$\rightarrow ABC$$

SOP expression for the output:

$$x = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

Designing Combinational Logic Circuits

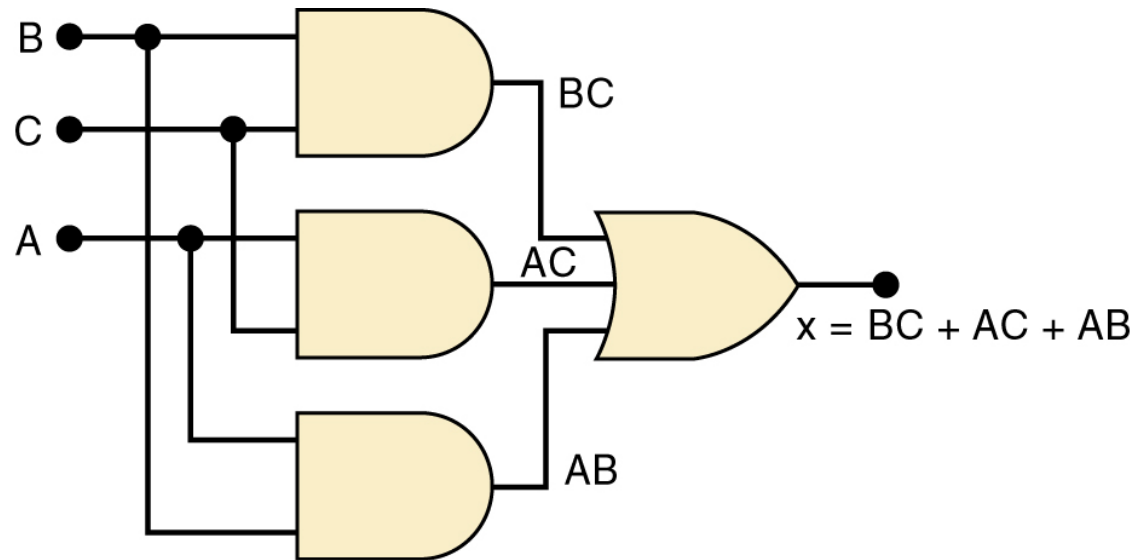
Design a logic circuit with three inputs, A, B, and C. Output to be HIGH only when a majority inputs are HIGH.

Simplified output expression:

$$x = BC + AC + AB$$

Implementing the circuit after factoring:

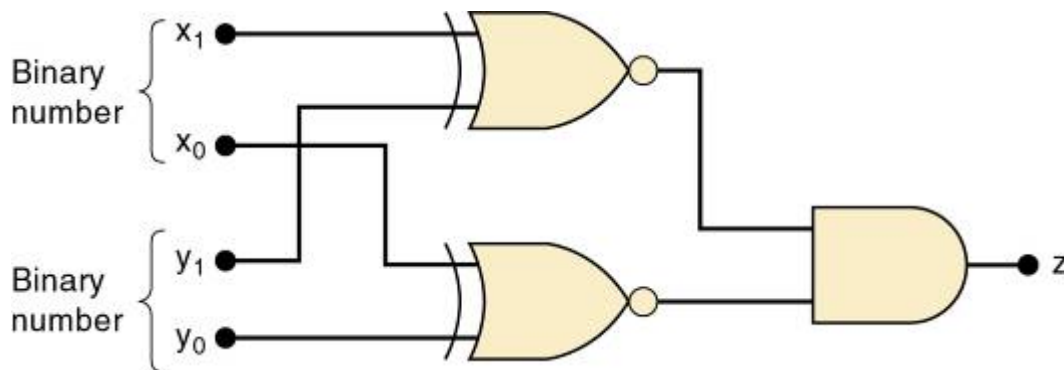
$$x = BC + AC + AB$$



Since the expression is in SOP form, the circuit is a group of **AND** gates, working into a single **OR** gate,

Example - 5 : Exclusive NOR Circuits

Truth table and circuit for detecting equality of two-bit binary numbers.



x_1	x_0	y_1	y_0	z (Output)
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Designing Combinational Logic Circuits

Example – 6 (Home Work)

- Design a logic circuit that is to produce a HIGH output when the voltage (represented by a four-bit binary number ABCD) is greater than 6V.

Designing Combinational Logic Circuits

Example – 7

- Four chairs A, B, C, and D are placed in a row. Each chair may be occupied (“1”) or empty (“0”). A Boolean function F is (“1”) if and only if there are two or more adjacent chairs that are empty.
 - a) Give the truth table defining the Boolean F .
 - b) Express F as a minterm expansion (standard sum of products).
 - c) Express F as a maxterm expansion (standard product of sums).
 - d) Simplify the minterm expansion of F .

Analysis procedure

To obtain the output Boolean functions from a logic diagram, proceed as follows:

1. Label all gate outputs that are a function of input variables with arbitrary symbols. Determine the Boolean functions for each gate output.
2. Label the gates that are a function of input variables and previously labeled gates with other arbitrary symbols. Find the Boolean functions for these gates.
3. Repeat the process outlined in step 2 until the outputs of the circuit are obtained.
4. By repeated substitution of previously defined functions, obtain the output Boolean functions in terms of input variables.

Example – 1

$$F_2 = AB + AC + BC; \quad T_1 = A + B + C; \quad T_2 = ABC; \quad T_3 = F_2' T_1;$$

$$F_1 = T_3 + T_2$$

$$F_1 = T_3 + T_2 = F_2' T_1 + ABC = A'BC' + A'B'C + AB'C' + ABC$$

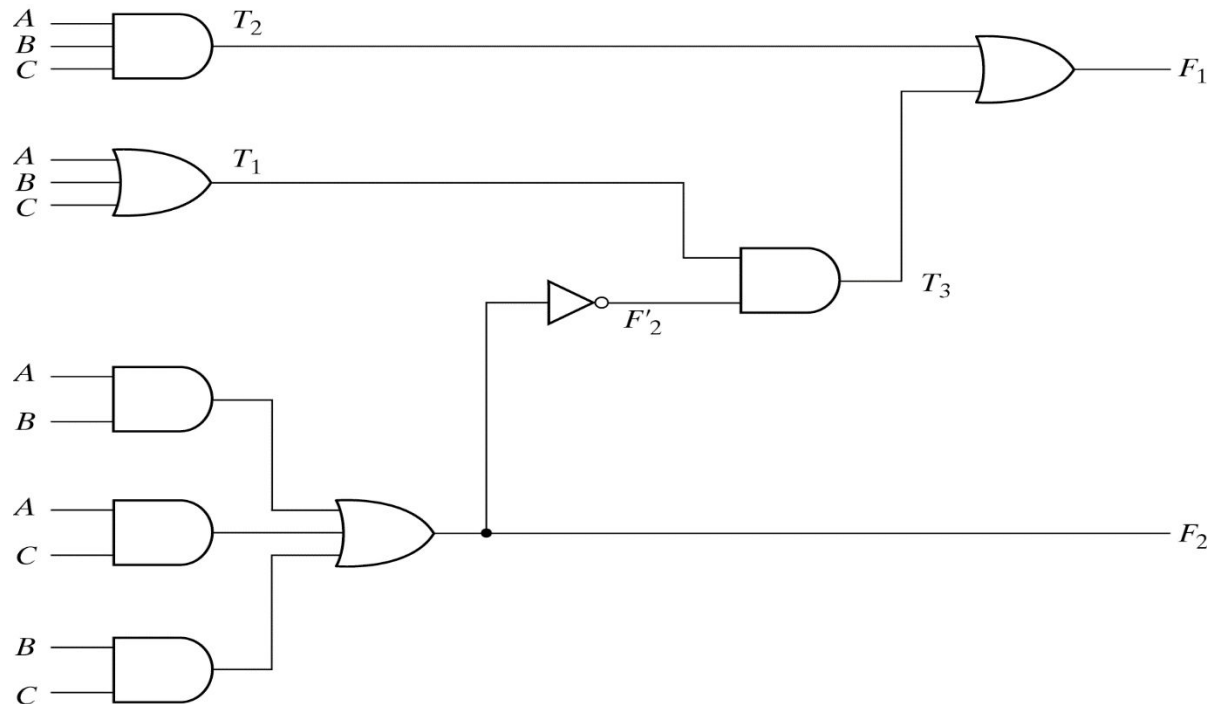


Fig. 4-2 Logic Diagram for Analysis Example

Derive truth table from logic diagram

- We can derive the truth table in Table 4-1 by using the circuit of Fig.4-2.

Table 4-1
Truth Table for the Logic Diagram of Fig. 4-2

<i>A</i>	<i>B</i>	<i>C</i>	<i>F</i> ₂	<i>F</i> ₂	<i>T</i> ₁	<i>T</i> ₂	<i>T</i> ₃	<i>F</i> ₁
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

Design procedure

1. Table 4-2 is a Code-Conversion example, first, we can list the relation of the BCD and Excess-3 codes in the truth table.

Table 4-2
Truth Table for Code-Conversion Example

Input BCD				Output Excess-3 Code			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Thank
You

A red rectangular button with a black border and a slight 3D effect.

Next topic